

Tallinna Tööstushariduskeskus

Noorem tarkvaraarendaja

ÕPPE- JA MÄNGURAKENDUS JÄÄTMETESORTIMISE ÕPPIMISEKS

(C# JA .NET MAUI ABIL)

Lõputöö

Juhendaja: *Marina Oleinik*

Koostaja: *Maria Smolina*

Tallinn 2026

AUTORIDEKLARATSIOON

Deklareerin, et käesolev lõputöö, mis on minu iseseisva töö tulemus, on esitatud Tallinna Tööstushariduskeskuse lõputunnistuse taotlemiseks noorem tarkvaraarendaja erialal.

Lõputöö alusel ei ole varem eriala lõputunnistust taotletud.

Autor: Maria Smolina *(allkirjastatud digitaalselt)*

Töö vastab kehtivatele nõuetele.

Juhendaja: Marina Oleinik *(allkirjastatud digitaalselt)*

SISUKORD

AUTORIDEKLARATSIOON.....	2
LÜHENDITE JA MÕISTETE SÕNASTIK.....	7
SISSEJUHATUS	10
1. TEOREETILINE OSA	11
1.1. Probleem ja valdkonna analüüs	11
1.2. Töö eesmärk ja ülesanded	12
1.3. Kasutatud tehnoloogiate valik ja põhjendus	13
1.4. Arendusmetoodika ja projekti planeerimine	14
2. SÜSTEEMI PROJEKTEERIMINE	17
2.1. Rakenduse arhitektuur	17
2.2. Kasutajaliidese kavandamine	18
2.3. Andmebaasi projekteerimine	21
3. PRAKTIILINE OSA.....	22
3.1. Rakenduse struktuur.....	22
3.1.1. Views	23
3.1.2. Models.....	24
3.1.3. ViewModels.....	24
3.1.4. Services	25
3.2. Peamised funktsioonid	25
3.2.1. Õpperežiim (Learning Mode)	26
3.2.2. Mängurežiim (Game Mode)	27
3.2.3. Jäätmete tuvastamine (Waste Detection)	28
3.2.4. Kasutaja statistika (Statistics)	29
3.2.5. Punktide ja edenemise süsteem.....	31
3.2.6. Seaded	33

3.3.	Algoritmid, funktsioonid ja loogika.....	34
3.3.1.	Mängurežiimi loogika.....	34
3.3.2.	Andmebaasi haldus	39
3.3.3.	Keelevahetuse loogika	41
3.3.4.	Google Cloud Vision API kasutamine (REST API).....	42
3.3.5.	Statistika ja graafikud (Syncfusion).....	46
4.	TESTIMINE	48
4.1.	Automaatne testimine (Unit testid).....	48
4.1.1.	Kasutajaprofiili testimine.....	48
4.1.2.	Mängurežiimi loogika testimine	49
4.1.3.	Tasemesüsteemi testimine	50
4.1.4.	Jäätmekategooriate vastendamise testimine	51
4.2.	Manuaalne testimine	53
5.	KASUTUSJUHEND	55
5.1.	Profiili seadistamine.....	55
5.2.	Mängurežiimi kasutamine.....	56
5.3.	Jäätmete tuvastamine	56
5.4.	Statistika vaatamine	56
5.5.	Seadete kasutamine	57
6.	EDASIARENDAMISE VÕIMALUSED.....	58
6.1.	Piirangud ja puudused.....	58
	KOKKUVÕTE	60
	KASUTATUD KIRJANDUS	61
	LISA A. Projekti planeerimine (Jira).....	64
	LISA B. Versioonihaldus (Sourcetree)	65
	LISA C. Figma Stiiliraamat	66
	LISA D. Lokaliseerimisfailid AppResources	67

JOONISTE LOETELU

Joonis 1. Taaskasutatud jäätmete kogus Euroopa Liidus 2022a. (Eurostat, 2024).....	11
Joonis 2. Kanban tahvel Jira's	15
Joonis 3. Jira EPIC nimekiri	16
Joonis 4. Värviskeem.....	19
Joonis 5. Visuaalsed elemendid	20
Joonis 6. Rakenduse andmebaasi ER-diagramm	21
Joonis 7. Rakenduses kasutatud MVVM arhitektuuri skeem	22
Joonis 8. Õpperežiimi (Learning Mode) kasutajaliides	26
Joonis 9. Mängurežiimi (Game Mode) kasutajaliides	27
Joonis 10. Jäätmete tuvastamise (Waste Detection) kasutajaliides	28
Joonis 11. Kasutaja statistika kuvamine rakenduses.....	29
Joonis 12. Statistika kuude ja päevade kaupa	30
Joonis 13. Punktide ja edenemise süsteem ning kasutajaprofiil	31
Joonis 14. Edenemise visuaalne kujutamine (taime kasv).....	32
Joonis 15. Kasutajaprofiili muutmise vaade	32
Joonis 16. Seadete ekraan.	33
Joonis 17. Tume teema näidis.....	33
Joonis 18. Koodilõik - kõigi prügikastide ja kõigi jäätmeliikide loend.....	35
Joonis 19. Koodilõik - kõikide jäätmeliikide loend, liigitatud tüüpide kaupa	36
Joonis 20. Rakenduses kasutatavate pildiresursside loend failis ImageResources.resx.....	37
Joonis 21. Koodilõik - vastuse kontrollimise funktsioon mängurežiimis.....	38
Joonis 22. Koodilõik - andmebaasi tabelite loomine (DatabaseService).....	39
Joonis 23. Koodilõik - kasutaja kogemuse (XP) uuendamise funktsioon	39
Joonis 24. Koodilõik - mänguseansi tulemuste salvestamine andmebaasi	40
Joonis 25. Koodilõik - jäätmeliigi statistika uuendamise funktsioon	40
Joonis 26. Koodilõik - keelevahetuse loogika realiseerimine (LanguageService)	42
Joonis 27. Koodilõik - keele valiku käsud ViewModel-is.....	42
Joonis 28. Koodilõik - Google Cloud Vision API teenuse initsialiseerimine.....	43
Joonis 29. Koodilõik - pildi analüüsi päringu koostamine ja saatmine	44
Joonis 30. Koodilõik - kasutajaprofiili testimine	48
Joonis 31. Koodilõik - mänguloogika testimise näited.....	49

Joonis 32. Koodilõik - tasemesüsteemi testimise näited.....	50
Joonis 33. Koodilõik - jäätmekategooriate määramise loogika testimine	51
Joonis 34. Koodilõik - testide käivitamise tulemused (Test Explorer).....	52

TABELITE LOETELU

Tabel 1. Rakenduse arhitektuurilised komponendid.....	17
Tabel 2. Rakenduse põhiekraanid	18

VIDEOTE LOETELU

Video 1. Mobiilirakenduse kasutusjuhend.....	55
--	----

LÜHENDITE JA MÕISTETE SÕNASTIK

API	<i>(Application Programming Interface)</i> on definitsioonide ja protokollide kogum, mis võimaldavad kahel tarkvarakomponendil omavahel suhelda. Teisisõnu võimaldavad API-d erinevatel tarkvarasüsteemidel omavahel rääkida. (DisainVeeb, 2022)
.NET MAUI	<i>(Multi-platform App UI)</i> on platvormiülene raamistik, mis võimaldab luua C# ja XAML-i abil natiivseid mobiili- ja töölauarakendusi. (Microsoft Learn, 2025)
C#	<i>(C-Sharp)</i> on Microsofti poolt välja töötatud programmeerimiskeel, mis töötab .NET Frameworki keskkonnas. C#-i kasutatakse veebirakenduste, töölauarakenduste, mobiilirakenduste, mängude ja palju muu loomiseks. (w3schools, 2022)
MVVM	<i>(Model-View-ViewModel)</i> on tarkvara arhitektuuriline lahendus, mis võimaldab eraldada graafilise kasutajaliidese arendamise - olgu see siis märgistuskeele või GUI-koodi abil - äriloojika või tagapõhja loojika (mudeli) arendamisest, nii et vaade ei sõltu ühestki konkreetsest mudeliplatvormist. (Wikipedia, 2020)
SQLite	C-keele raamatukogu, mis rakendab väikest, kiiret, iseseisvat, väga töökindlat ja kõiki funktsioone hõlmavat SQL-andmebaasi mootorit. (SQLite, 2019)
UI	<i>(User Interface)</i> on ühenduslüli kasutaja ja arvutiprogrammi vahel. Kasutajaliides teeb programmi funktsionaalsuse kasutajale kättesaadavaks. (Wikipedia, 2010)
UX	<i>(User Experience)</i> kogemus, mida klient või kasutaja saab toote, süsteemi või teenusega suheldes. (IBM, 2025)

XAML	<i>(Extensible Application Markup Language)</i> on deklaratiivne märgistuskeel. .NET-programmeerimismudelil kasutamisel lihtsustab XAML .NET-rakenduse kasutajaliidese loomist. (Learn Microsoft, 2025)
Git	versioonihaldussüsteem, mis on loodud selleks, et hallata kiiresti ja tõhusalt nii väikeseid kui ka väga suuri projekte. (Git, 2024)
GitHub	omandipõhine arendajate platvorm, mis võimaldab arendajatel oma koodi luua, salvestada, hallata ja jagada. (Wikipedia, 2019)
Jira	Atlassiani arendatud tarkvaratoode, mis võimaldab veade ja probleemide jälgimist ning paindlikku projektijuhtimist. (Wikipedia, 2019)
Kanban	Agile-raamistik, mis visualiseerib tööd, piirab pooleliolevate tööde hulka ja soodustab pidevat täiustamist läbipaistvate töövoogude kaudu. (Radigan, 2024)
EPIC	Agile-metoodika raames koostatud mahukad tööd, mis on jagatud väiksemateks kasutajate lugudeks järkjärguliseks valmimiseks. (Atlassian, 2024)
Unit testing	koodilõik, mis kontrollib väiksema, eraldiseisva rakenduskoodi lõigu - tavaliselt funktsiooni või meetodi - õigsust. Üksiktesti eesmärk on kontrollida, kas koodilõik töötab ootuspäraselt, vastavalt arendaja poolt selle loomiseks kasutatud teoreetilisele loogikale. (AWS, 2024)
XP	<i>(Experience Points)</i> on mõõtühik, mida kasutatakse mõnedes lauamängulistes rollimängudes (RPG) ja rollimängulistes videomängudes, et väljendada mängija tegelase elukogemust ja arengut mängu käigus. (Wikipedia, 2025)

JSON	<i>(JavaScript Object Notation)</i> on tekstipõhine formaat andmete salvestamiseks ja vahetamiseks viisil, mis on nii inimesele loetav kui ka masinloetav. (Erickson, 2022)
Base64	sarnaste binaar-tekst-kodeerimisskeemide rühm, mis esitavad binaarandmeid ASCII-stringi vormingus, teisendades need 64-kohalise arvusüsteemi esitusviisiks. (MDN Contributors, 2025)
HTTP	<i>(Hypertext Transfer Protocol)</i> rakenduskihi protokoll hüpermeedia dokumentide, näiteks HTML-i edastamiseks. See on loodud veebibrauserite ja veebiserverite vaheliseks suhtluseks, kuid seda saab kasutada ka muudel eesmärkidel, näiteks masinate vahelises suhtluses, API-dele programmilise juurdepääsu võimaldamisel ja muudes valdkondades. (MDN Contributors, 2019)
Agile	Agile-metoodika on lähenemisviis, mis jagab töö etappideks, rõhutades pidevat tulemuste esitamist ja täiustamist. Agile pakub meeskondadele eeliseid, võimaldades paindlikku planeerimist, kiiret elluviimist ja pidevat hindamist, mis viib paindlikumate ja edukamate tulemusteni. (Atlassian, 2025)
REST API	<i>(Representational State Transfer)</i> on arhitektuurstiil, mis määratleb veebiteenuste loomisel kasutatavate piirangute komplekti. REST API määratleb toimingute komplekti, mida saab ressursidega teha, ja kasutab standardmeetodeid, nagu GET, POST, PUT ja DELETE. (DisainVeeb, 2022)
SVG	<i>(Scalable Vector Graphics, 'skaleeruv vektorgraafika')</i> XML-põhine vektorgraafikaformaat kahemõõtmelise graafika kujutamiseks, mis toetab interaktiivsust ja animatsiooni. (Wikipedia, 2019)

SISSEJUHATUS

Planeedi saastumine ja keskkonna seisundi halvenemine on tingitud inimtegevuse erinevatest aspektidest. Rahvastiku kasv, teaduse ja tehnika areng ning planeedi industrialiseerimine on peamised põhjused, mis avaldavad negatiivset mõju meie planeedi ökoloogilisele seisundile. Atmosfääri saastumine, muldade ja taimekasvatuse halvenemine, jäätmete tekkimine ja uute haiguste esinemine on vaid mõned probleemid, mis tekivad keskkonna negatiivse mõju tagajärjel. (Wang & Zhou, 2021)

Jäätmete tõhusaks töötlemiseks on vaja mitte ainult sobivat infrastruktuuri, vaid ka seda, et kasutajad mõistaksid sorteerimise reegleid. Paljudel on raskusi jäätmete liigi kindlaksmääramisel ja õige konteineri valimisel, mis vähendab protsessi üldist tõhusust. (Minelgaitė & Liobikienė, 2019)

Käesoleva lõputöö raames on kavas luua mobiilirakendus „SortIt“, mis õpetab kasutajaid jäätmeid õigesti sorteerima. Nimi on valitud nii, et see peegeldaks kohe selle olemust - sorteerimise õpetamist. See on lühike, kergesti meelde jääv ja otseselt seotud rakenduse funktsioonidega.

Peamine ülesanne on töötada välja interaktiivne vahend, mis selgitab jäätmete sorteerimise reegleid arusaadavalt, võimaldab kontrollida kasutaja teadmisi ja aitab kujundada püsivaid keskkonnasäästlikke harjumusi. Selleks kasutatakse mängulisi elemente, mis suurendavad motivatsiooni ja kaasatust. (Majuria, Koivisto, & Hamari, 2018)

Selle eesmärgi saavutamiseks on kavas lisada rakendusse mängurežiim, statistika- ja edusammude süsteem ning võimalus jäätmete tuvastamiseks telefoni kaamera abil. See muudab rakenduse kasutamise mugavamaks ning õppimise arusaadavamaks, tõhusamaks ja interaktiivsemaks.

Projekti planeerimisetapis valiti arendamiseks programmeerimiskeel C# ja märgistuskeel XAML, kasutades .NET MAUI raamistikku. Selline valik tuleneb asjaolust, et eelnimetatud tehnoloogiad toetavad platvormidevahelist ühilduvust, mugavat töötamist rakenduse loogikaga ja MVVM-arhitektuuri, mille tõttu muutub kood loetavamaks ning seda on lihtsam edaspidi muuta ja testida.

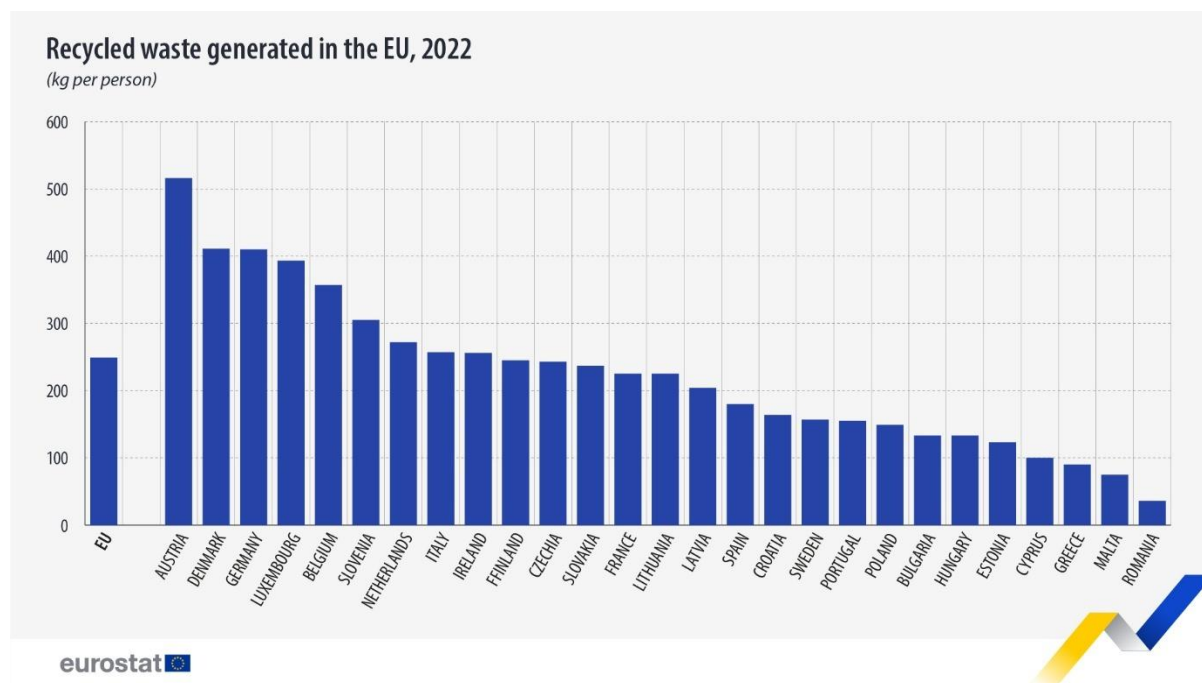
Arendustöö käigus kasutatakse lähtekoodi ja testide haldamiseks [GitHubi](#) platvormi, mis tagab arendusprotsessi läbipaistvuse ja võimaldab jälgida muudatusi koodibaasis.

1. TEOREETILINE OSA

1.1. Probleem ja valdkonna analüüs

Jäätmete hulk on muutumas üheks peamiseks keskkonnaprobleemiks. Euroopa riikides on täheldatav keskkonnale kahjulike olmejäätmete koguse suurenemine.

Euroopa Liidu riikides tekib [Eurostati](#) 2022. aasta andmete kohaselt keskmiselt 249 kg töödeldavaid jäätmeid iga elaniku kohta aastas. Selle näitaja on Euroopa eri piirkondade riikides erinev. (Eurostat, 2024)



Joonis 1. Taaskasutatud jäätmete kogus Euroopa Liidus 2022a. (Eurostat, 2024)

Hoolimata sellest, et mitmes maailma riigis, sealhulgas Eestis, on kasutusele võetud jäätmete liigiti kogumise süsteemid, ei ole nende tõhusus piisav. Üks sellise ebapiisava tõhususe põhjuseid on elanikkonna vähene teadlikkus kehtivatest jäätmete liigiti kogumise eeskirjadest. Kogu elanikkond võib olla ebakindel selles osas, millisesse konteinerisse tuleb teatavat liiki jäätmed visata.

Sorteerimise kvaliteeti mõjutavad ka käitumuslikud tegurid. Näiteks võivad elanikkonna harjumused või motivatsioon sorteerimise kvaliteeti oluliselt mõjutada. Traditsioonilised

elanikkonna teavitamise viisid - juhised või plakatid - võivad osutuda ebaefektiivseks, eriti noorema põlvkonna puhul. (Linder, Giusti, Samuelsson, & Barthel, 2021)

Uued uuringud näitavad, et digitaalsete lahenduste, eelkõige mobiilirakenduste kasutamine võib oluliselt suurendada kasutajate kaasatust. Enamik kasutajaid kasutab tänapäeval aktiivselt nutitelefone igapäevaelus, mis tähendab, et mobiilirakendused on õppevahenditena väga nõutud. (Statista, 2025)

Üks tõhus lähenemisviis on „gamification“ - mänguelementide lisamine mittemängulistesse süsteemidesse. Selline lähenemisviis võimaldab suurendada kasutajate motivatsiooni ja parandada teabe omandamist. (Majuria, Koivisto, & Hamari, 2018)

Seega võib praeguste jäätmete sorteerimissüsteemide probleemide tõttu rääkida vajadusest võtta kasutusele täiendavaid lahendusi, mis suurendavad teadlikkust ja kaasatust ning pakuvad kasutajatele abi. (Linder, Giusti, Samuelsson, & Barthel, 2021)

Üks võimalikke lahendusi sellele probleemile on hariduslike funktsioonidega ja jäätmete tuvastamise võimalusega mobiilirakenduse loomine.

1.2. Töö eesmärk ja ülesanded

Käesoleva lõputöö eesmärk on luua hariduslik mobiilirakendus-mäng jäätmete sorteerimise õpetamiseks, kasutades programmeerimiskeelt C# ja platvormi .NET MAUI.

Väljatöötatud mobiilirakendus on mõeldud selleks, et aidata kasutajatel õppida jäätmete sorteerimise reegleid, kontrollida oma teadmisi mängulises vormis ning määrata jäätmete liiki, tuvastades need kaamera pildil.

Selle eesmärgi saavutamiseks seati järgmised ülesanded:

1. Valdkonna analüüs ja jäätmete sorteerimisega seotud probleemid.
2. Rakenduse ja selle funktsionaalsuse nõuete määratlemine.
3. Rakenduse arendamiseks vajalike tehnoloogiate ja tööriistade valik.
4. Rakenduse andmebaasi arhitektuuri ja struktuuri määratlemine.
5. Rakenduse programmeerimine C# ja .NET MAUI abil.
6. Jäätmete sorteerimist käsitleva mängulise õppemooduli programmeerimine.
7. Prügi tuvastamise funktsiooni rakendamine.

8. Graafikute ja statistika kuvamise rakendamine.
9. Rakenduse testimine ja vigade parandamine.

Lõputöö tulemusena peab valmima toimiv mobiilirakendus, mida saab kasutada jäätmete sorteerimise õpetamiseks.

1.3. Kasutatud tehnoloogiate valik ja põhjendus

Rakenduse arendamiseks valiti programmeerimiskeeleks **C#** ja raamistikuks **.NET MAUI**. .NET MAUI suurim eelis on see, et see põhineb C#-il. Lisaks võimaldab see raamistik luua rakendusi erinevatele operatsioonisüsteemidele, nagu iOS, Android, Windows ja macOS. (Microsoft, 2023)

Üks .NET MAUI raamistiku kasutamise lisaväärtusi on mugavate sisseehitatud tööriistade olemasolu võrgukommunikatsiooniks (klass **HttpClient**), mis hõlbustavad suhtlemist **REST API**-ga, autentimist ja HTTP-pealkirjade haldamist. Samuti on võimalus kasutada andmebaase nagu **SQLite** ja **Entity Framework Core**, mis muudavad kohalike ja kaugandmebaaside arendamise lihtsamaks. (Calisir, 2024)

.NET MAUI võimaldab muid täiustatud valikuid failisüsteemi käsitlemiseks, eriti kuna seal on sisseehitatud klassid, sealhulgas **File**, **Stream** ja **Path**, ning samuti on olemas platvormispetsiifilised lisad seadme kettaruumi liideseks. (Calisir, 2024)

Kasutajaliidese kujundamiseks valiti **XAML**, mis võimaldab struktureerida kasutajaliidese kirjeldust ning toetab projekti arhitektuuri selget eristamist rakenduse loogikast.

Peamiseks arenduskeskkonnaks valiti **Microsoft Visual Studio**, mis sisaldab C# ja .NET MAUI kasutamiseks vajalikke tööriistu, nagu näiteks: silumistööriistad, kasutajaliidese kujundaja ja integratsioon versioonihaldussüsteemiga.

Kasutajate andmete, mängutulemuste ja statistika salvestamine on realiseeritud **SQLite** abil - sisseehitatud andmebaasi abil, mis on optimeeritud mobiilirakenduste jaoks ega vaja eraldi serveri paigaldamist.

Statistika ja graafikute visualiseerimiseks kasutati raamatukogu **Syncfusion**, mis pakub vajalikke valmiskomponente graafikute, diagrammide ja andmete visualiseerimise loomiseks ja haldamiseks. See lahendus lihtsustab arendusprotsessi ning parandab rakenduse välimust.

Prügi tuvastamise funktsioon on rakendatud **Google Cloud Vision API** abil, täpsemalt öeldes **Label Detection** mehhanismi abil. See teenus analüüsib pilte ja tuvastab neil olevaid objekte masinõppe tehnoloogiate abil.

Projekti juhtimine toimus süsteemi [Jira](#) abil, kus loodi Kanban-tahvel ülesannete planeerimiseks ja arendusprotsessi jälgimiseks. See võimaldas tööd struktureerida ja projekti etappideks jagada. (Atlassian, 2026)

Versioonide haldamiseks ja koodi hoidmiseks kasutati platvormi GitHub ning muudatuste jälgimiseks kasutati programmi [Sourcetree](#).

Rakenduse „SortIt“ lähtekood on kättesaadav [GitHubi repositooriumis](#).

Valitud tehnoloogiate rakendamise tulemusena töötati välja funktsionaalne mobiilirakendus, mis ühendab endas andmete salvestamise, statistika visualiseerimise ja piltide intelligentset tuvastamist.

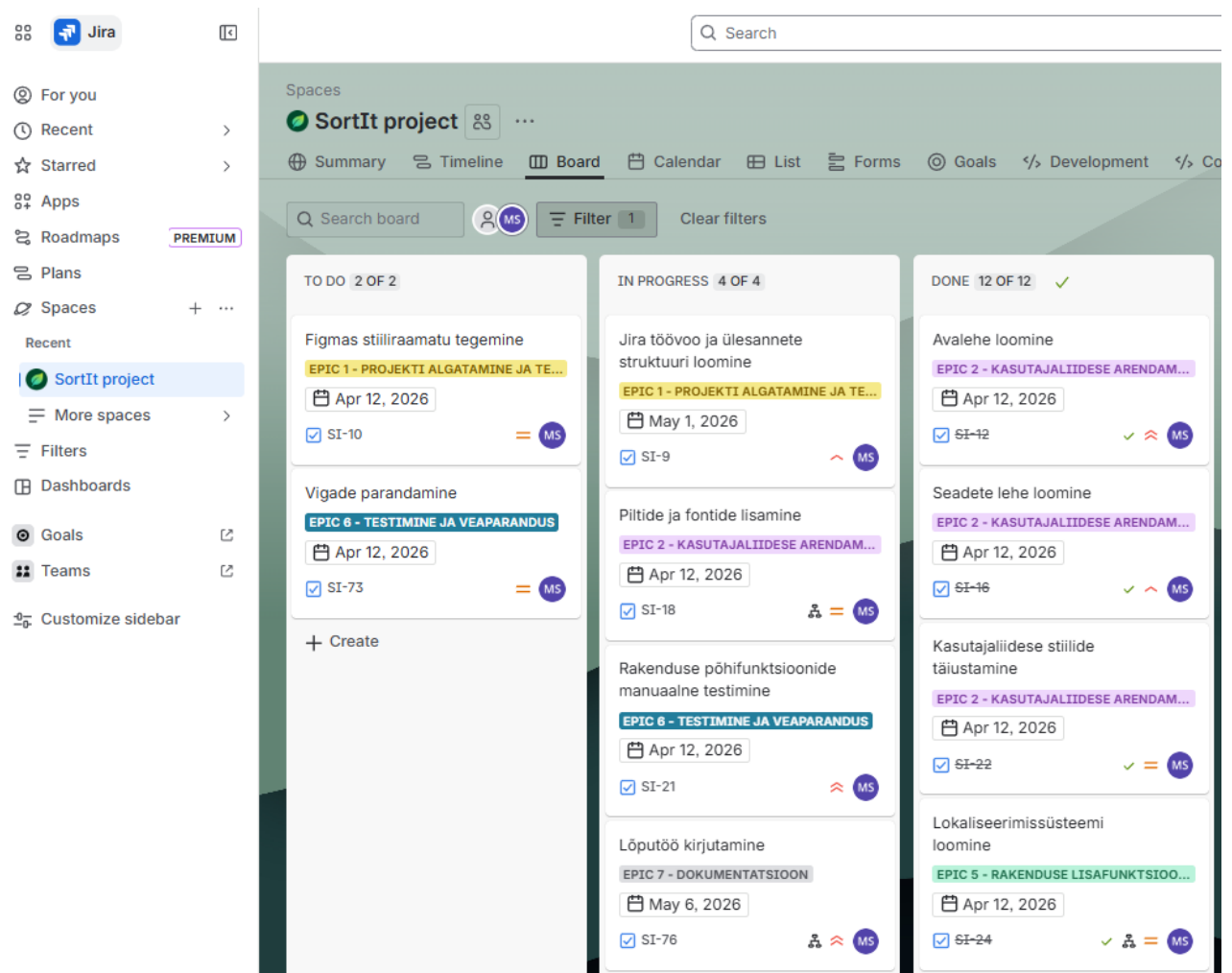
1.4. Arendusmetoodika ja projekti planeerimine

Rakenduse arendamisel kasutati Agile'iga sarnast paindlikku metoodikat. Projekt viidi ellu etapiviisiliselt, väikeste iteratsioonidena, laiendades järk-järgult funktsionaalsust.

Arendusprotsess jaguneb järgmistesse etappidesse:

- Probleemi analüüs ja ülesande sõnastamine
- Kasutajaliidese ja andmebaasi struktuuri kavandamine
- Rakenduse põhiarhitektuuri väljatöötamine
- Õpperežiimi ja mängurežiimide rakendamine
- Google Cloud Vision API integreerimine
- Statistika ja graafikute lisamine
- Testimine ja vigade parandamine
- Dokumentide ettevalmistamine ja lõputöö kirjutamine

Ülesannete planeerimiseks kasutati Jira süsteemi Kanban-tahvlit.



Joonis 2. Kanban tahvel Jira's

Kõik projekti ülesanded jagati EPIC-kategooriatesse:

- **EPIC 1** - Projekti algatamine ja tehniline ettevalmistus
- **EPIC 2** - Kasutajaliidese arendamine
- **EPIC 3** - Andmebaas ja andmehaldus
- **EPIC 4** - Mänguloogika arendamine
- **EPIC 5** - Rakenduse lisafunktsioonid
- **EPIC 6** - Testimine ja veaparandus
- **EPIC 7** - Dokumentatsioon

☐	Work	Status	Due date	Assignee
☐	🔗 SI-74 EPIC 7 - Dokumentatsioon	IN PROGRESS ▾	May 06, 2026	 Maria Smolina
☐	🔗 SI-63 EPIC 6 - Testimine ja veaparandus	IN PROGRESS ▾	Apr 12, 2026	 Maria Smolina
☐	🔗 SI-55 EPIC 5 - Rakenduse lisafunktsioonid	DONE ▾	Apr 12, 2026	 Maria Smolina
☐	🔗 SI-53 EPIC 4 - Mänguloogika arendamine	DONE ▾	Nov 09, 2025	 Maria Smolina
☐	🔗 SI-23 EPIC 3 - Andmebaas ja andmehaldus	DONE ▾	Apr 12, 2026	 Maria Smolina
☐	🔗 SI-11 EPIC 2 - Kasutajaliidese arendamine	DONE ▾	Apr 12, 2026	 Maria Smolina
☐	🔗 SI-1 EPIC 1 - Projekti algatamine ja tehniline ettevalmistus	DONE ▾	May 06, 2026	 Maria Smolina
+ Create				7 of 7 ↺

Joonis 3. Jira EPIC nimekiri

Ülesanded liikusid veergude vahel: „To Do“, „In Progress“ ja „Done“. See võimaldas jälgida projekti edenemist ja saada ülevaate sellest, millised ülesanded on juba täidetud, millised on pooleli ja millised on veel tegemata.

Ülesannete üksikasjalikum struktuur ja planeerimisprotsess on kättesaadav Jira projektikeskkonnas - [„SortIt“ Jira Project](#)

Lisaks on projekti arendusetappide ajakava (timeline) esitatud lisas ([LISA A](#)).

Samuti kasutati arendustöös versioonihaldussüsteemi Sourcetree ([LISA B](#)), mis võimaldas säilitada projekti muudatuste ajalugu ja töötada turvalisemalt.

Projekti planeerimine ja Kanban-tahvli kasutamine võimaldasid arendusprotsessi korraldada, ülesandeid ühtlaselt jaotada ja projekti edukalt lõpule viia.

2. SÜSTEEMI PROJEKTEERIMINE

2.1. Rakenduse arhitektuur

Arendatav rakendus on .NET MAUI abil loodud mobiilirakendus. Rakendus töötab kliendi poolel ning suhtleb väliste teenuste ja kohaliku andmebaasiga.

Rakenduse arhitektuuri võib jagada järgmistesse tasanditesse:

Kasutajaliides (UI)	See kiht vastutab teabe kuvamise eest kasutajale ja rakendusega suhtlemise eest.
Rakenduse loogika	See tasand vastutab kasutaja tegevuste töötlemise, mängus antud vastuste kontrollimise, punktide arvestamise, pildituvastuse tulemuste töötlemise, statistika haldamise ja kasutajaandmete haldamise eest.
Andmetega töötamine	See tasand vastutab andmete salvestamise ja andmebaasist laadimise eest. Siin on rakendatud klassid, mis tegelevad kasutajate, mängutulemuste, statistika ja seadistustega.
Välised teenused (Google Cloud Vision API)	Piltide tuvastamiseks kasutatakse Google Cloud Vision API-d. Rakendus saadab pildi API-le ja saab REST-arhitektuuri abil nimekirja pildil leitud objektidest.
Andmebaas (SQLite)	Andmete salvestamiseks kasutatakse SQLite-andmebaasi, mis asub kasutaja seadmes kohalikult.

Tabel 1. Rakenduse arhitektuurilised komponendid

2.2. Kasutajaliidese kavandamine

Enne kasutajaliidese väljatöötamise alustamist tuli läbi mõelda rakenduse ekraanide struktuur ja nende vaheline navigeerimine. Peamine eesmärk oli luua lihtne ja arusaadav kasutajaliides, mis oleks mugav kasutajatele igas vanuses.

Rakenduse peamised ekraanid:

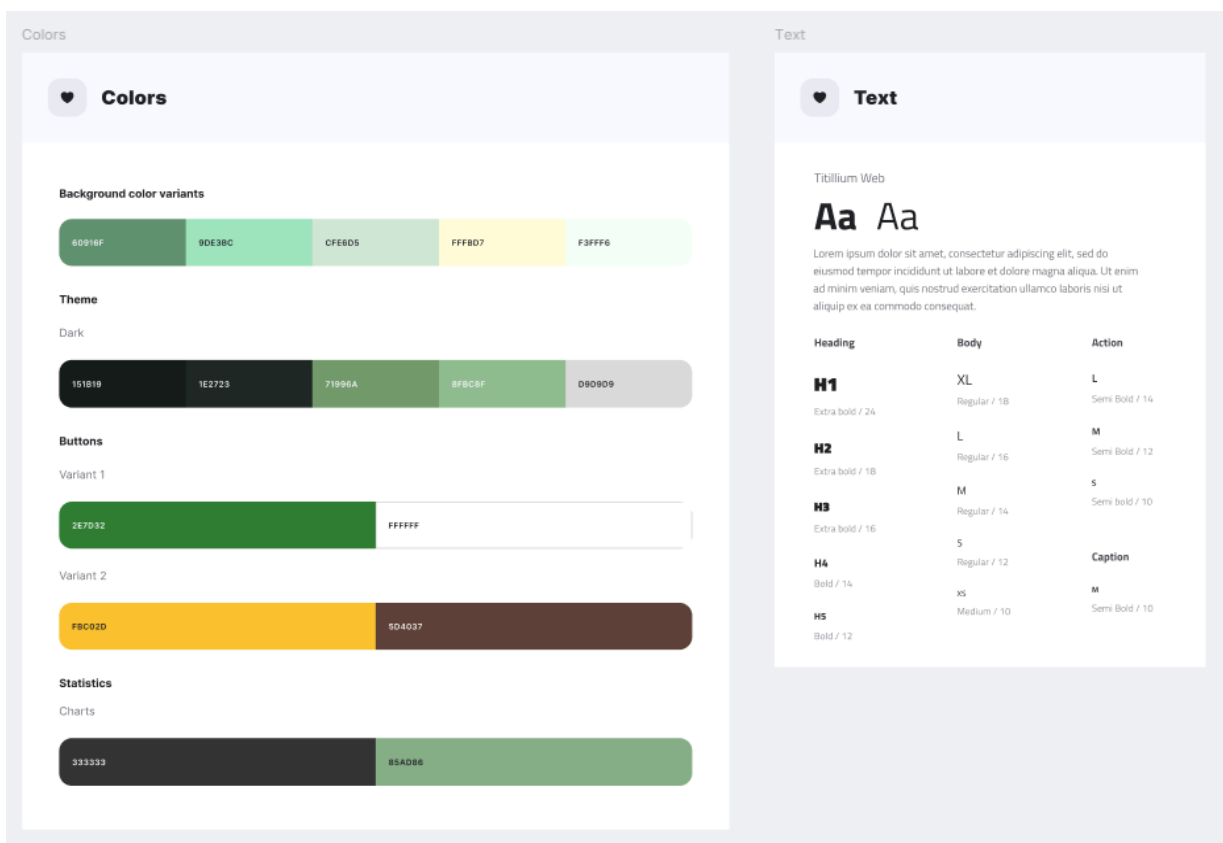
Isiklik profiil	Kasutaja saab vaadata statistikat, kogemusi, taset ja auastet. Ta saab muuta oma nime ja profiilipilti.
Navigatsioon	Ekraanide vahel liikumine on realiseeritud .NET MAUI navigeerimissüsteemi abil.
Õpperežiim (Learning Mode)	Selles režiimis tutvub kasutaja erinevate jäätmeliikidega ja saab teada, millisesse konteinerisse neid tuleb visata.
Mängurežiim (Game Mode)	Kasutaja peab jäätmed õigesti konteineritesse sorteerima ja saab selle eest punkte.
Prügi tuvastamine (Camera Detect)	Kasutaja teeb objektist foto, mille järel rakendus määrab jäätmeliigi Google Cloud Vision API abil.
Statistika (Statistics)	Sellel ekraanil kuvatakse graafikuid, kasutaja tulemusi, punktisummat ja õigete vastuste arvu.
Seaded (Settings)	Selles jaotises saab kasutaja seadistada rakenduse parameetreid, näiteks valida kasutajaliidese keele, lülitada heli sisse või välja ning valida kujundusteema.

Tabel 2. Rakenduse põhiekraanid

Enne kasutajaliidese väljatöötamise alustamist loodi rakenduse disainiprototüüp tööriistas [Figma](#). Projekteerimise käigus koostati stiiljuhend, mis hõlmas värvipaletti, tüpograafiat, kasutajaliidese elemente ja rakenduse üldist visuaalset stiili ([LISA C](#)).

Kasutajaliidese elemendid, nagu illustratsioonid ja logo, on joonistatud käsitsi vektorgraafika abil, samas kui navigeerimisikoonid on pärit veebisaidilt [Flaticon](#) ja prügikastide piktogrammide veebisaidilt [Liigitikogumine](#). Kõik kasutajaliidese elemendid on eksporditud Figma-st SVG-vormingus.

SVG-failid on vektorpõhised, mis tähendab, et neid saab sujuvalt mis tahes suuruseks muuta ilma kvaliteeti kaotamata. SVG-vorming võimaldab kasutada sama graafilist elementi erinevates mõõtmetes ilma kvaliteedikadudeta, mistõttu sobib see hästi skaleeritavate kasutajaliidese komponentide loomiseks. Lisaks on SVG-failid sageli väiksemad, neid on lihtsam hallata ja neid saab koodis dünaamiliselt kohandada - värve või suurusi saab muuta lennult. (Banwo, 2025)

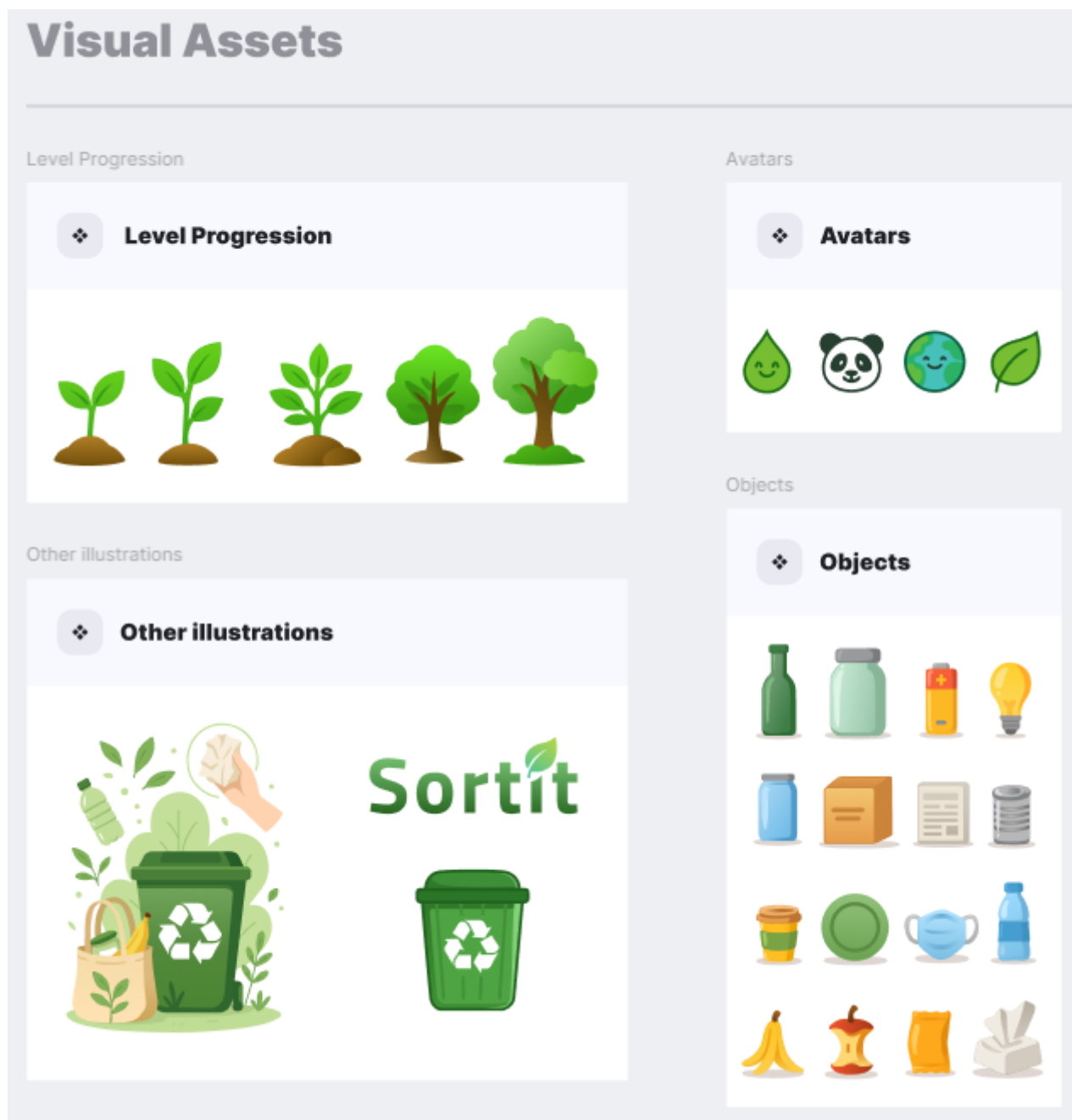


Joonis 4. Värviskeem

Figma kasutamine võimaldas eelnevalt läbi mõelda ekraanide välimuse, tagada kasutajaliidese ühtse stiili ja lihtsustada disaini edasist rakendamist rakenduses.

Rakenduse prototüübiga saab tutvuda lingi kaudu - [„SortIt“ Stiiliraamat](#).

Peamised ekraanid asuvad vahekaardil „Templates“, illustratsioonid ja ikoonid vahekaardil „Components“ ning tüpograafia vahekaardil „Styles“.



Joonis 5. Visuaalsed elemendid

2.3. Andmebaasi projekteerimine

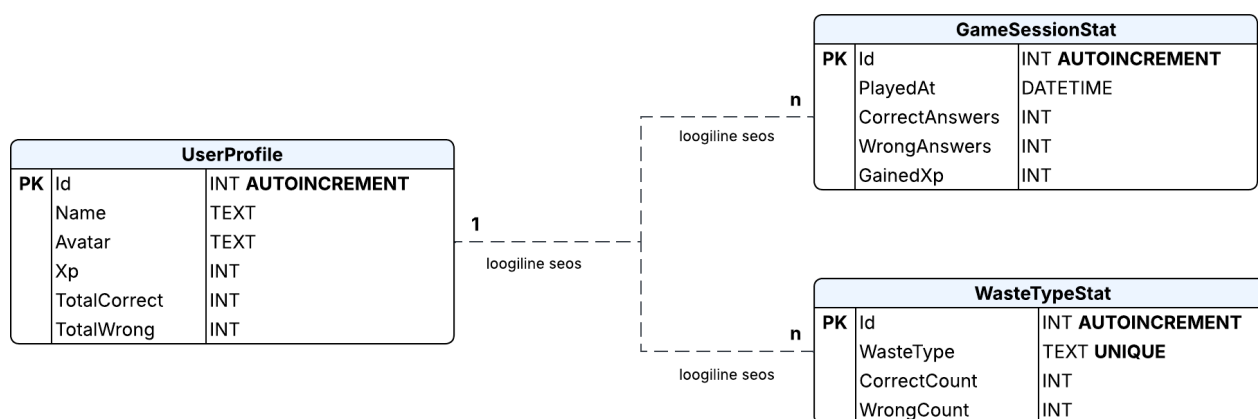
Kasutajate andmete, mängutulemuste ja statistika salvestamiseks loodi andmebaas. Andmebaasiks valiti SQLite, kuna see sobib hästi mobiilirakendustele ega vaja eraldi serverit.

Andmebaas salvestab:

- kasutajad
- mängude tulemused
- statistika
- jäätmeliigid
- jäätmekategooriad
- kasutaja punktid

Joonisel on esitatud rakenduse andmebaasi struktuur, mis hõlmab järgmisi põhitabeleid: UserProfile, GameSessionStat ja WasteTypeStat. Tabel UserProfile sisaldab kasutaja profiiliandmeid ja tema üldist edusamme. Tabel GameSessionStat salvestab teavet mängusessioonide tulemuste kohta, sealhulgas õigete ja valede vastuste arvu ning saadud kogemuspunkte (XP). Tabel WasteTypeStat on mõeldud jäätmeliikide statistika salvestamiseks.

Tabelitevahelised seosed on loogilist laadi ja rakenduvad rakenduse tasandil. Ühel kasutajal võib olla mitu mängusessiooni ja statistilist kirjet, kuid praeguses andmebaasi rakenduses välisvõtmeid ei kasutata. Diagramm on loodud tööriista [Lucidchart](#) abil.



Joonis 6. Rakenduse andmebaasi ER-diagramm

3. PRAKTILINE OSA

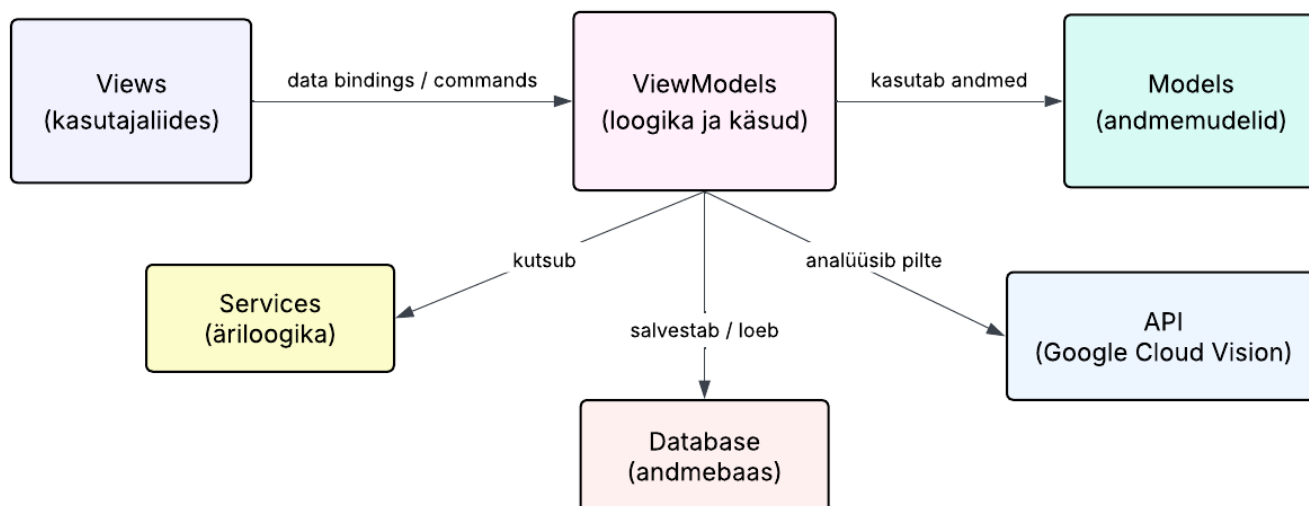
3.1. Rakenduse struktuur

Rakenduse arhitektuur kavandati vastavalt tänapäevastele platvormiülestele mobiilsüsteemide loomise põhimõtetele.

Väljatöötatud rakenduse projekt koosneb järgmistest põhiosadest:

- Views (*kasutajaliidese ekraanid*)
- Models (*andmemudelid*)
- Services (*rakenduse teenused ja äriloojika*)
- ViewModels (*liidese ja andmete koostoime loogika*)
- Database (*andmebaasiga seotud toimingud*)
- API (*suhtlemine Google Cloud Vision API-ga*)
- Resources (*pildid, ikoonid, stiilid, lokaliseerimine*)

Joonisel 7 on esitatud rakenduse struktuuri diagramm, mis kajastab arhitektuuri põhiliste komponentide koostoimet. Diagramm on koostatud tööriistas [Lucidchart](#).



Joonis 7. Rakenduses kasutatud MVVM arhitektuuri skeem

Rakenduse arendamisel kasutati MVVM-arhitektuuri (Model-View-ViewModel), mis tagab kasutajaliidese, äriloogika ja rakenduse andmete eraldatuse. Selline lähenemine rakendab süsteemi komponentide vahelise vastutuse eraldamise põhimõtet (separation of concerns), mille kohaselt täidab iga rakenduse osa rangelt määratletud funktsiooni. See aitab vähendada moodulitevahelist seotust, parandada koodi loetavust ja lihtsustada tarkvaratoote hooldamist. (Sommerville, 2016, lk 170-175)

Mängurežiimi realiseerimisel kasutati täiendavalt **GameService** klassi, kuhu eraldati mängu äriloogika. See võimaldas hoida **GameViewModeli** kergemana ning eraldada mängureeglid kasutajaliidese loogikast.

Lisaks vastab projekti struktuur modulaarse arhitektuuri põhimõtetele, mille kohaselt peab süsteem koosnema sõltumatutest komponentidest, millel on selgelt määratletud ülesanded. Selline lähenemisviis hõlbustab süsteemi üksikute osade testimist, koodi taaskasutamist ja rakenduse funktsionaalsuse edasist laiendamist. (Sommerville, 2016, lk 175-180)

3.1.1. Views

View vastutab kasutajaliidese eest. See koosneb XAML-i abil loodud rakenduse lehtedest (ContentPage). View kuvab kasutajale andmeid ja edastab kasutaja toimingud ViewModelile.

Views-i projekti kuuluvad:

- EditProfilePage (*kasutajanime ja avatari muutmise aken*)
- GamePage (*interaktiivse õppe mängurežiimi ekraan*)
- GuidePage (*ekraan, millel on teave jätmete sorteerimise eeskirjade kohta*)
- MainPage (*avakuva rakenduse esmakordsel käivitamisel*)
- ProfilePage (*profili ekraan, kus kuvatakse tase, kogemus ja statistika*)
- SettingsPage (*rakenduse seadete ekraan*)
- StatisticsPage (*statistika ja graafikute kuvamise ekraan*)
- WasteDetectionPage (*ekraan, kus kaamera abil määratakse jäätmeliik*)

Vaade ei sisalda äriloogikat, vaid kuvab ainult andmeid ja töötleb kasutajaliidese sündmusi.

3.1.2. Models

Mudel on rakenduse andmemudel. Mudeleid kasutatakse andmete salvestamiseks, mis laaditakse andmebaasist või mida rakenduses kasutatakse.

Projekti Models alla kuuluvad:

- Statistics (*Statistika*)
 - ChartPoint (*punktimudel graafikute joonistamiseks*)
 - GameSessionStat (*mängusessiooni tulemuste salvestamise mudel*)
 - WasteTypeChartPoint (*andmemudel jäätmeliikide graafiku jaoks*)
 - WasteTypeStat (*jäätmeliikide statistika mudel*)
- WasteModels (*jäätmete sorteerimise loogikaga seotud mudelite rühm*)
 - Bin (*jäätmete sorteerimise konteinerimudel*)
 - SortableItem (*sorteeritava objekti mudel*)
 - WasteType (*jäätmeliigi mudel*)
- RoundResult (*mänguvooru tulemuse mudel*)
- Slide (*õppeslaidi elemendi mudel*)
- Theme (*rakenduse kujundusteema mall*)
- UserProfile (*kasutaja profiili mudel koos andmete ja edusammudega*)

Mudelid sisaldavad ainult andmete omadusi ega sisalda kasutajaliidese loogikat.

3.1.3. ViewModels

ViewModel on View ja Model vaheline ühenduslüli. See sisaldab kogu rakenduse põhilist loogikat, määrab käsud ja töötleb kasutajaga suhtlemist, laadib andmeid ning vastutab suhtlemise eest teenustega.

ViewModel saab andmeid mudelitest ja teenustest, töötleb neid ja saadab need kuvamiseks View-le. Lisaks kasutab ViewModel käsked, mida saab käivitada nuppude vajutamise või muude kasutaja toimingute abil.

Projektis on esitatud järgmised ViewModels:

- GameViewModel (*mängurežiimi kasutajaliidese loogika ja kasutaja tegevuste vahendamine*)
- GuideViewModel (*õppematerjalide kuvamine*)
- LanguageViewModel (*rakenduse keele vahetamise haldamine*)
- ProfileViewModel (*kasutaja profiili ja andmetega töötamine*)
- StatisticsViewModel (*andmete ettevalmistamine statistika kuvamiseks*)
- WasteDetectionViewModel (*jäätmeliigi määramise loogika pildi põhjal*)

3.1.4. Services

Services sisaldavad rakenduse ärilooikat ja abifunktsioone. ViewModel kasutab neid ülesannete täitmiseks, mis ei nõua kasutaja otsest osalemist kasutajaliideses, näiteks andmebaasiga suhtlemine, andmete töötlemine või suhtlemine välisteenustega.

Projektis kuuluvad Services alla:

- AudioService (*rakenduse helidega töötamine*)
- CloudVisionAPIService (*API piltide tuvastamiseks*)
- GameService (*mängurežiimi ärilooika ja mängureeglite haldamine*)
- DatabaseService (*töö SQLite-andmebaasiga*)
- LanguageService (*keele lokaliseerimine ja vahetamine*)
- LevelService (*kasutaja taseme ja edusammude arvutamine*)
- SlideService (*andmete esitamine õppematerjalide jaoks*)
- WasteCategoryMapper (*jäätmeliikide ja konteinerite vastavustabel*)

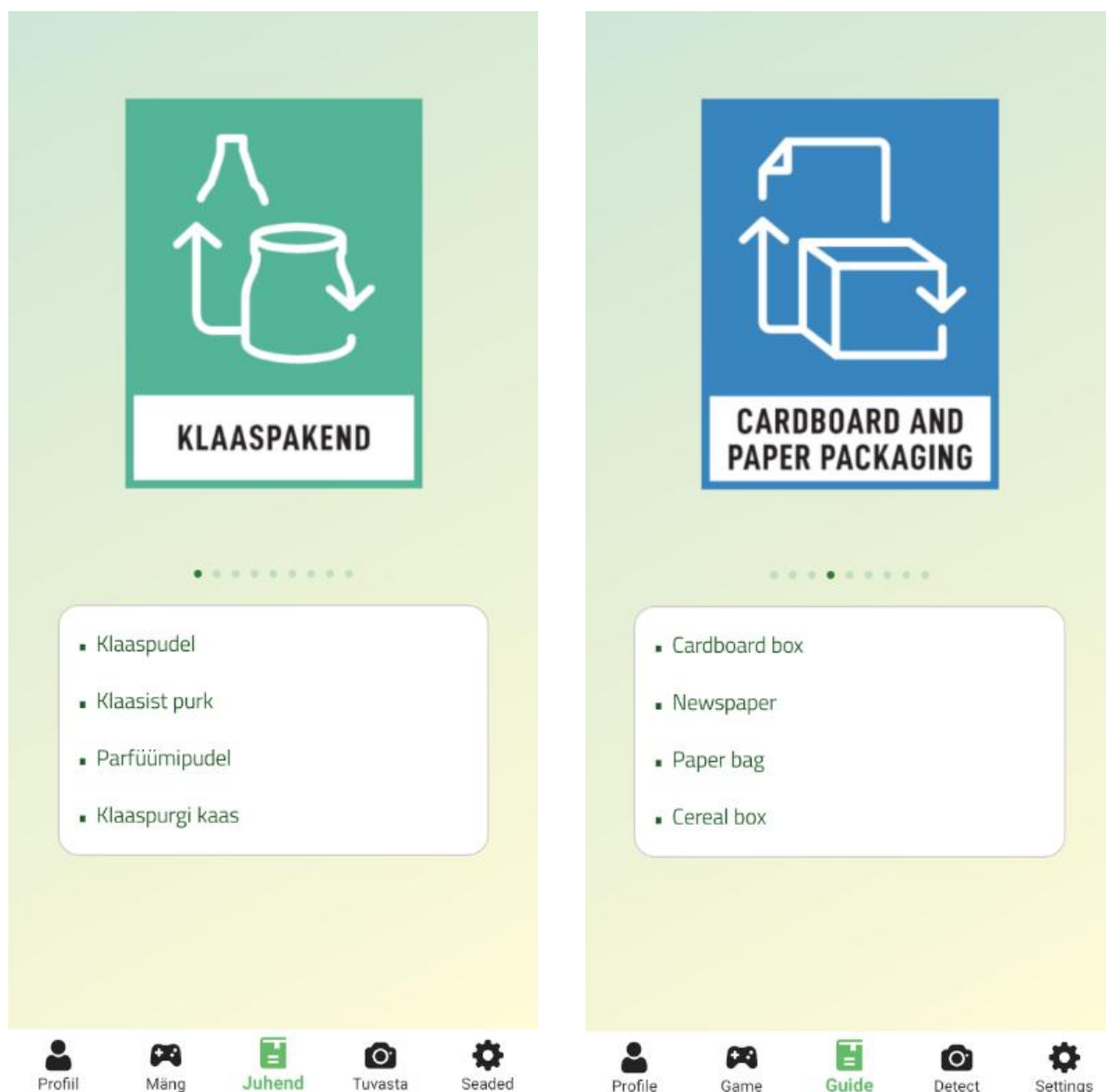
3.2. Peamised funktsioonid

Rakendus sisaldab mitut põhilist funktsiooni, mis võimaldavad õppimist, teadmiste kontrollimist mängulises vormis, jäätmete tuvastamist ja kasutaja statistika kuvamist. Järgnevalt vaadeldakse iga nimetatud funktsiooni üksikasjalikumalt.

3.2.1. Õpperežiim (Learning Mode)

Õpperežiim on rakenduse hariduslik alus, mis võimaldab kasutajatel tutvuda erinevate jäätmekategooriate ja nende sorteerimise eeskirjadega. Selles režiimis on esitatud erinevate jäätmeliikide pildid ja kirjeldused, et oleks selge, milline jäätmekonteiner on sobiv.

Visuaalsete materjalidena kasutatakse piktogramme, mis on võetud veebilehelt [Liigitikogumine](#), mis pakub tasuta juurdepääsu jäätmete sorteerimise materjalidele. Need piktogrammid põhinevad Taani sorteerimissüsteemil ja on kohandatud kasutamiseks Eestis. (Liigitikogumine, 2025)



Joonis 8. Õpperežiimi (Learning Mode) kasutajaliides

3.2.2. Mängurežiim (Game Mode)

Rakenduse peamiseks interaktiivseks funktsiooniks on mängurežiim, kus kasutajad peavad ajapiiranguga sorteerima jäätmed õigetesse konteineritesse. Ekraanil kuvatakse jäätmete pilt ja mitu konteinerit. Kasutaja peab valima jäätmete liigile vastava õige konteineri.

Statilise harjutuse asemel soodustab mäng kogemuslikku õppimist; pärast iga katset antakse kohest tagasisidet, premeerides õigeid valikuid punktidega ja märkides ära vead. See reaajas toimiv tagasisidesüsteem aitab parandada info meeldejätmist ja suurendada kasutaja aktiivset osalust.

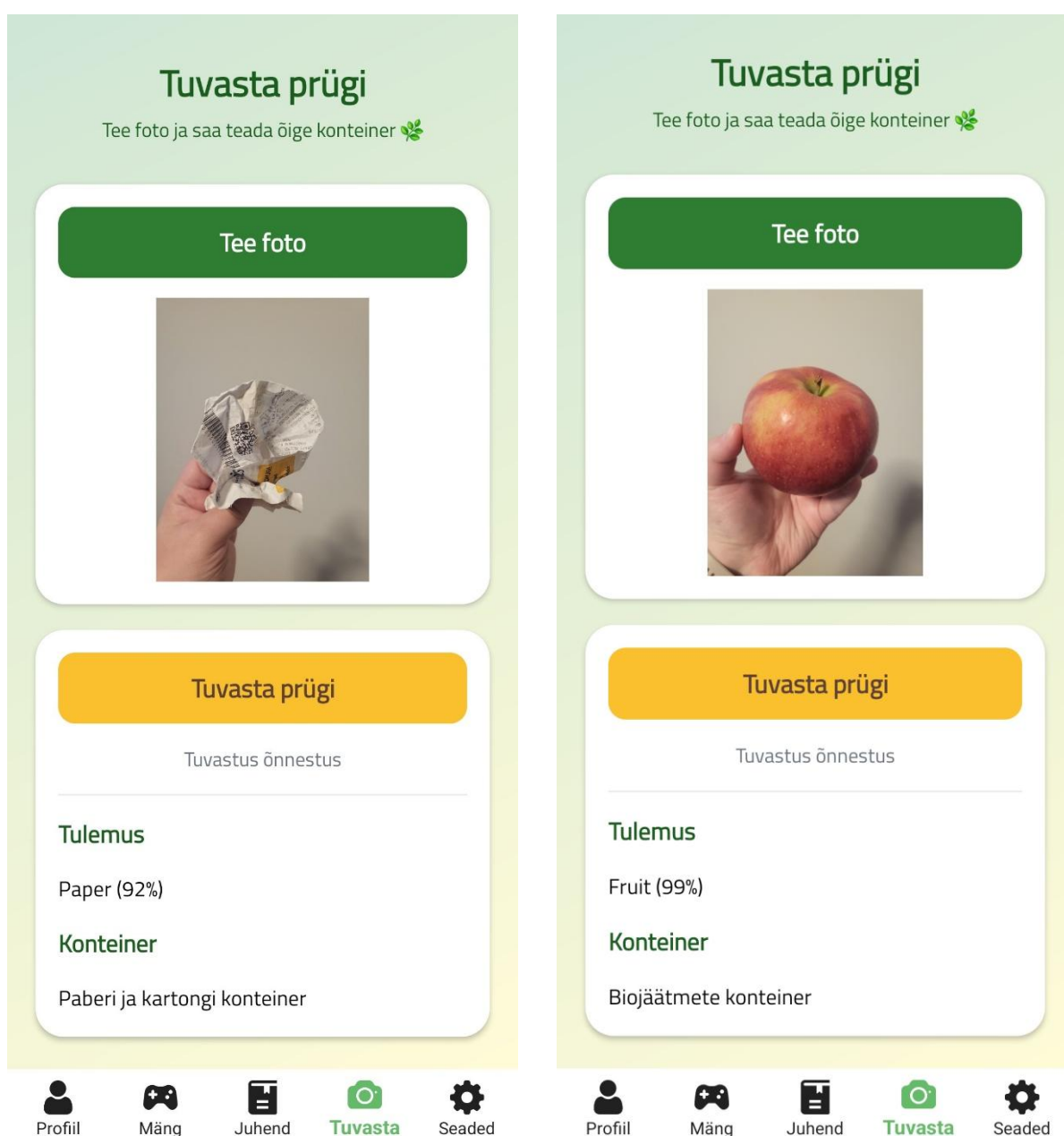


Joonis 9. Mängurežiimi (Game Mode) kasutajaliides

3.2.3. Jäätmete tuvastamine (Waste Detection)

Üks rakenduse huvitavaid funktsioone on võimalus tuvastada jäätmed foto põhjal. Kasutajad saavad pildistada mis tahes eseme ning rakendus määrab kindlaks jäätmeliigi ja soovitab õige jäätmekonteineri.

Tuvastamisfunktsioon muudab rakenduse mitte ainult õppimisvahendiks, vaid ka praktiliseks tööriistaks. Kasutaja saab seda igapäevaelus kasutada, et kiiresti kindlaks teha, kuhu konkreetne ese tuleks visata.



Joonis 10. Jäätmete tuvastamise (Waste Detection) kasutajaliides

3.2.4. Kasutaja statistika (Statistics)

Rakendus salvestab kasutaja tulemused ning kuvab statistikat tema õppimise ja mängimise kohta. See võimaldab kasutajal jälgida oma edusamme ja analüüsida tulemusi.

Kasutajaliides kuvab neid andmeid diagrammide abil, mis on loodud raamatukogu **Syncfusion** abil.

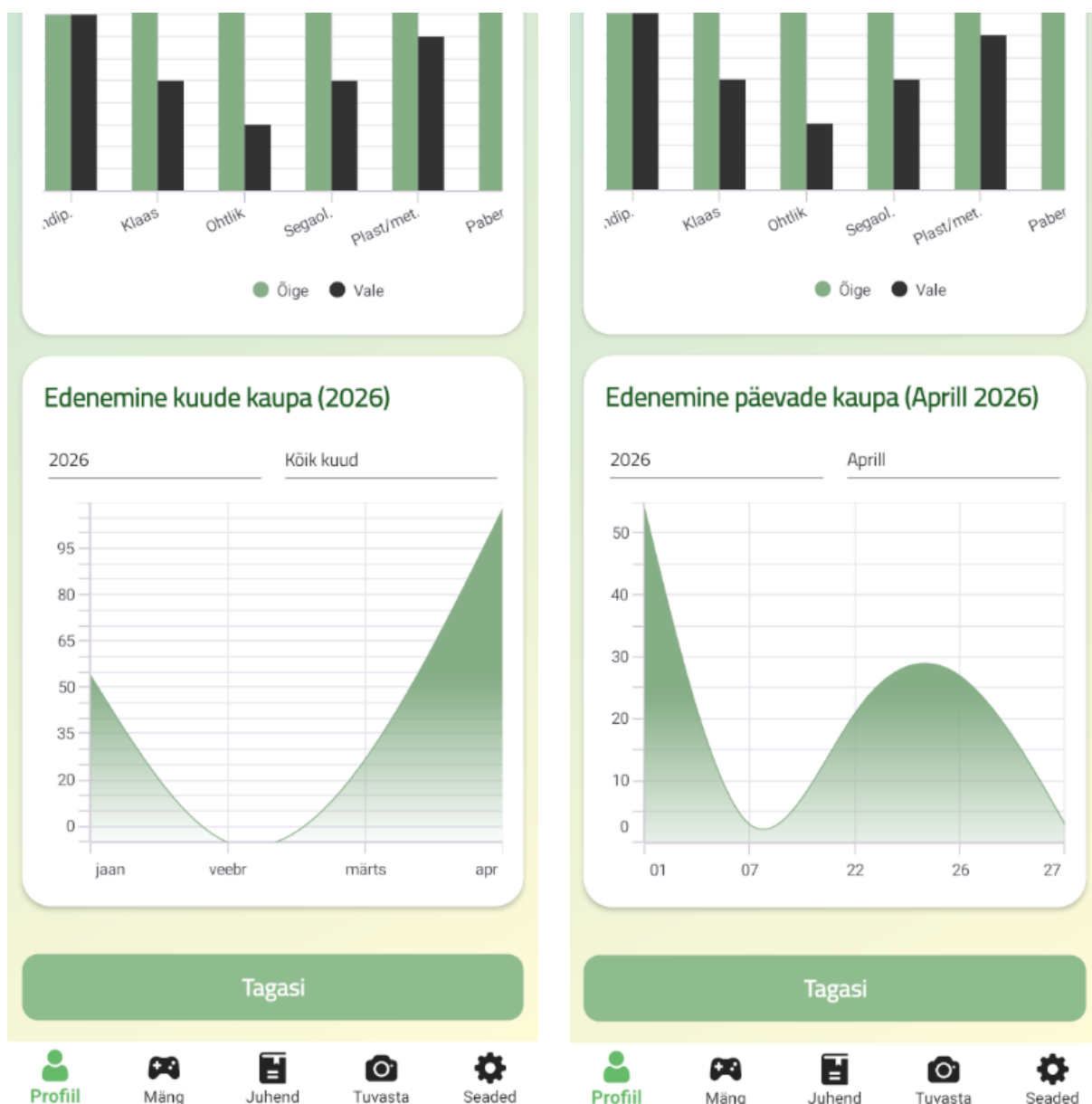


Joonis 11. Kasutaja statistika kuvamine rakenduses

Statistikas kuvatakse järgmised näitajad:

- punktide koguarv (XP)
- õigete vastuste arv
- valede vastuste arv
- mängitud mängude koguarv
- kasutaja edusammud

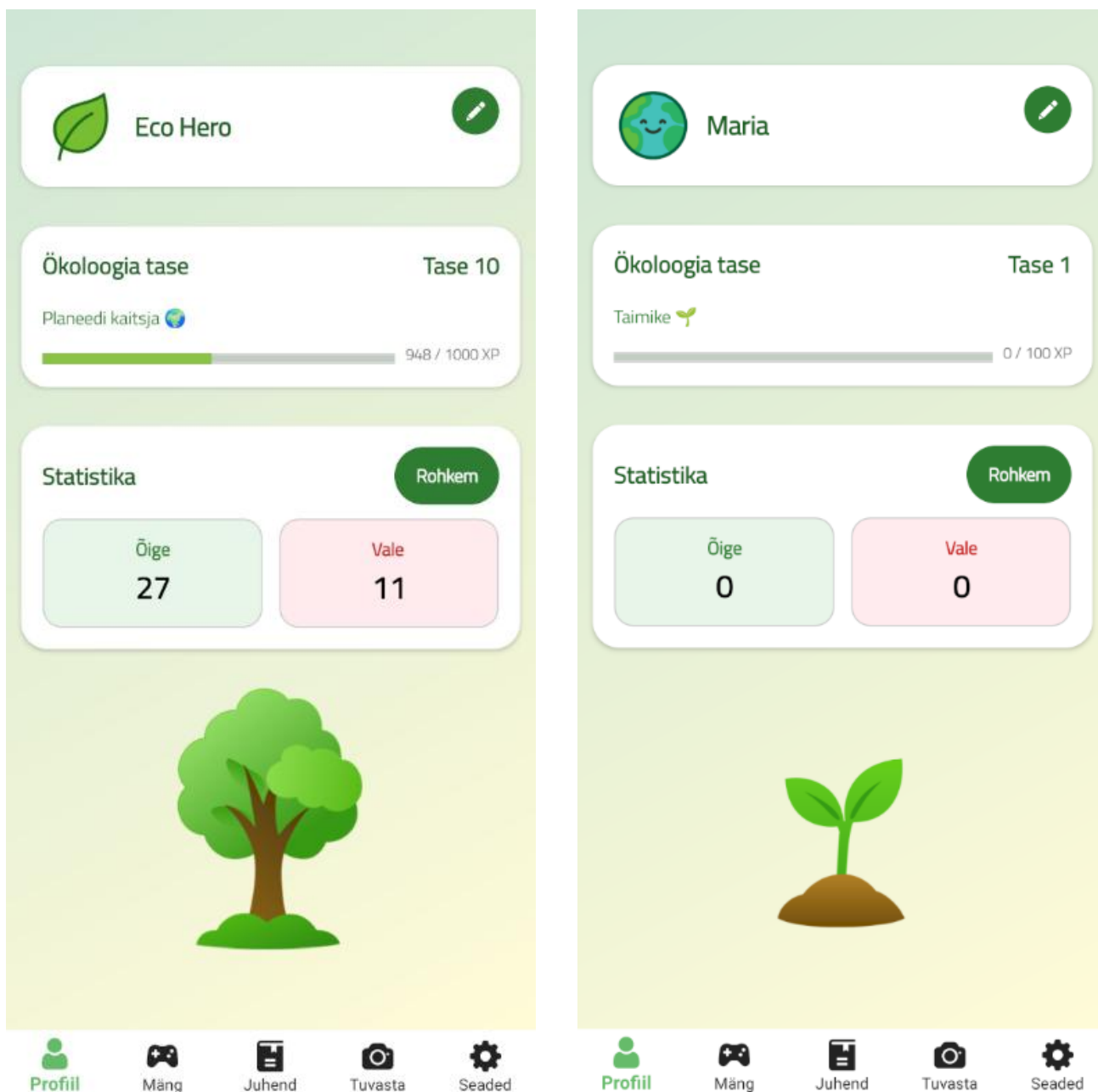
Need arvud uuendatakse täpsuse tagamiseks automaatselt pärast iga mänguvooru. Selge visuaalne ülevaade saavutustest ja arengust parandab kasutajale tulemuste jälgitavust ning suurendab õppimisprotsessi kaasahaaravust.



Joonis 12. Statistika kuude ja päevade kaupa

3.2.5. Punktide ja edenemise süsteem

Mängustamine on rakenduse disaini keskne element, mille punktide ja tasemete süsteem premeerib edukat mängimist. Õiged vastused annavad teatud hulga XP-punkte, mis kogunevad kõrgemate tasemete avamiseks. Punktide kasvades muutub ka mängija auaste.



Joonis 13. Punktide ja edenemise süsteem ning kasutajaprofiil

Lisaks kasutatakse rakenduses dünaamilist visuaalset elementi: animeeritud taime, mille suurus ja keerukus kasvavad vastavalt kasutaja taseme tõusule. See graafiline edusammude näitamine muudab preemiasüsteemi visuaalselt mitmekesisemaks.



Joonis 14. Edenemise visuaalne kujutamine (taime kasv)

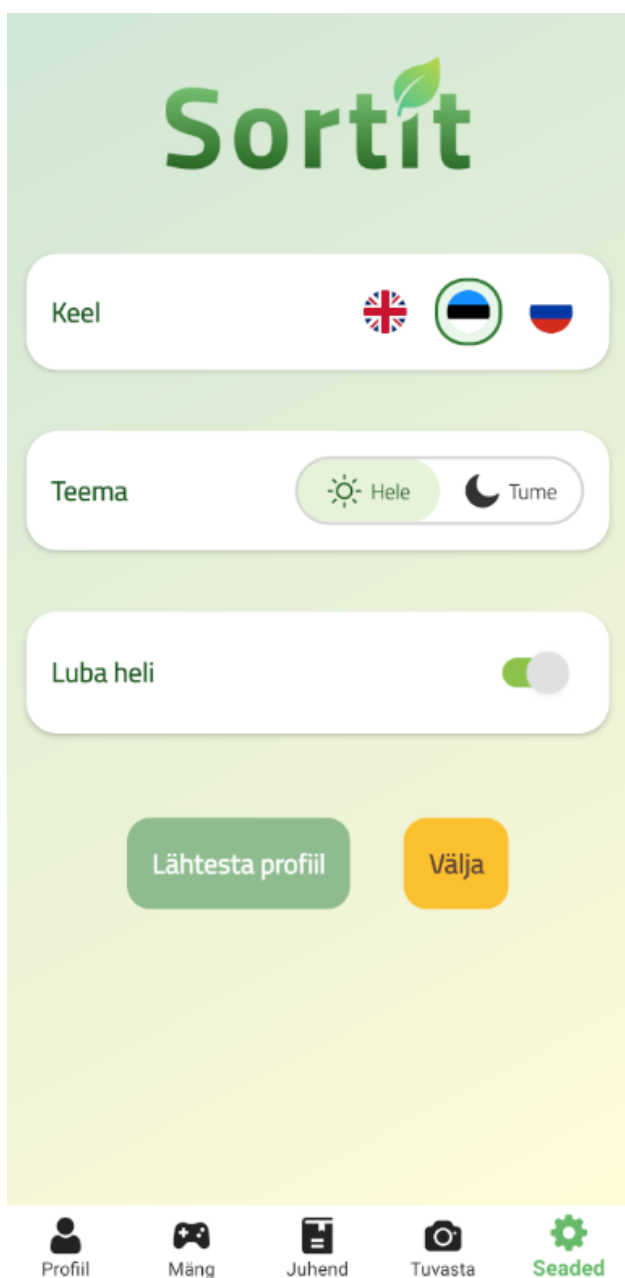
Lisaks on kasutajal võimalus oma profiili kohandada. Rakenduses on võimalik muuta kasutajanime ja valida avatari. Valikus on mitu avatari, mis võimaldab kasutajal rakendust isikupärastada ja muuta seda enda maitse järgi.

A screenshot of a user profile editing interface. At the top, the text 'Mängija nimi' is displayed above a text input field containing the name 'Maria'. Below this, the text 'Vali avatar' is displayed above four avatar options: a green leaf, a blue and green globe with a smiling face (which is highlighted with a green border), a black and white panda face, and a green water drop with a smiling face. At the bottom of the interface are two buttons: 'Salvesta' (Save) and 'Tühista' (Cancel).

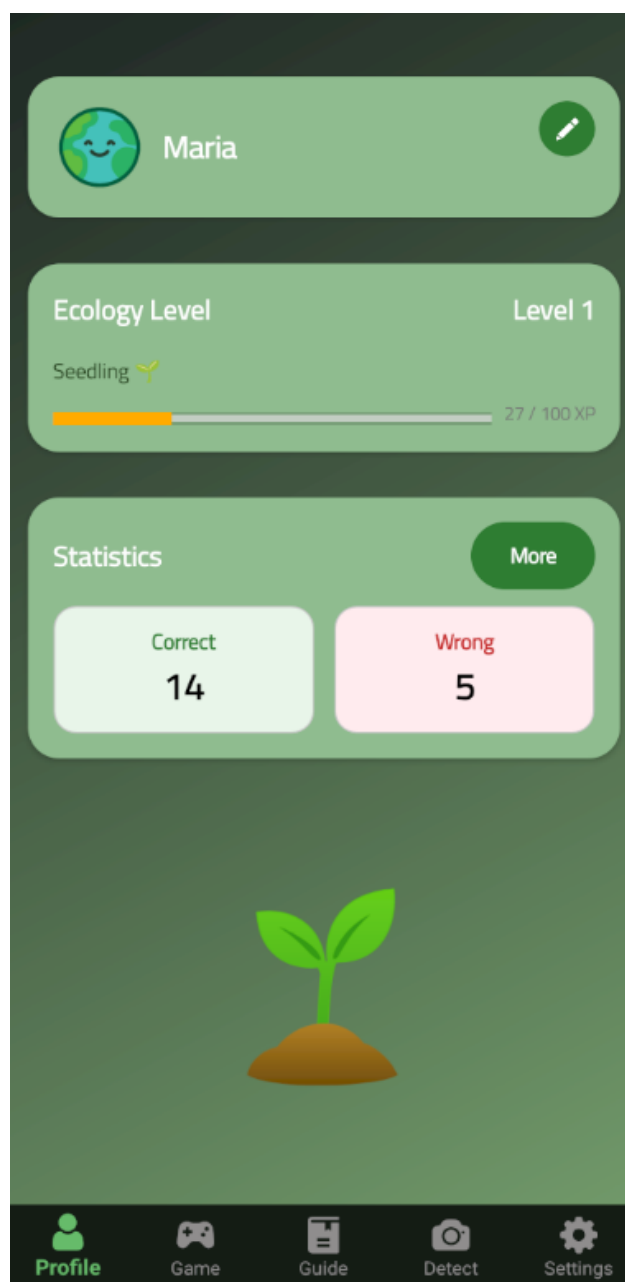
Joonis 15. Kasutajaprofiili muutmise vaade

3.2.6. Seaded

Rakenduses on olemas seadete menüü, kus kasutaja saab muuta rakenduse keelt inglise, eesti või vene keeleks, vahetada heleda ja tumeda teema vahel ning aktiveerida või deaktiveerida heliefekte. Need seaded tagavad, et rakendust saab täielikult kohandada vastavalt isiklikele eelistustele, mis aitab parandada üldist kasutuskogemust.



Joonis 16. Seadete ekraan.



Joonis 17. Tume teema näidis

3.3. Algoritmid, funktsioonid ja loogika

Käesolevas jaotises käsitletakse rakenduse peamisi funktsioone ja nende tööpõhimõtteid. Kuna rakendus on arendatud MVVM-arhitektuuri alusel, paikneb kasutajaliidese sidumisloogika ViewModelides ning äri loogika teenusekihis (Services). Selline lähenemine võimaldab hoida ViewModelid kergemad ning eraldada mängureeglid ja andmetöötluse kasutajaliidest. Projekt koosneb järgmistest kaustadest: **Models**, **Services**, **ViewModels** ja **Views**.

Rakenduse olulisemateks komponentideks on:

- GameViewModel
- GuideViewModel
- ProfileViewModel
- StatisticsViewModel
- WasteDetectionViewModel
- GameService
- DatabaseService
- CloudVisionAPIService
- LevelService
- LanguageService

Komponentide vaheline suhtlus toimub andmete sidumise (data binding) mehhanismi abil, mis võimaldab kasutajaliidest automaatselt uuendada, kui ViewModelis muutuvad andmed.

Rakenduse teenused vastutavad äri loogika täitmise ja suhtlemise eest väliste süsteemidega, nagu SQLite andmebaas ja Google Cloud Vision API. ViewModels on kasutajaliidese ja teenuste vaheliseks ühenduslülilik, mis edastab nende vahel andmeid ja kasutaja toiminguid.

3.3.1. Mängurežiimi loogika

Mängurežiim on üks rakenduse peamisi funktsioone. Mängurežiimis on ülesannete genereerimise aluseks kaks sisemist andmekogumit, mis on realiseeritud sõnastikena (Dictionary). Esimene sõnastik, **allBins**, sisaldab täielikku nimekirja jäätmete sorteerimiseks

kättesaadavatest konteineritest. Iga **WasteType** loendi väärtuse jaoks määratakse Bin-objekt, milles hoitakse konteineri tüüpi ja vastavat pilti. Selle teabe abil on võimalik saada vajalik konteiner koos vastava pildiga, mida sorteerimise ajal ekraanil kuvada.

Allpool on esitatud sõnastiku allBins realiseering, mis sisaldab kõigi mängurežiimis kasutatavate jäätmekonteinerite loendit koos neile vastavate pildiresurssidega:

```
private readonly Dictionary<WasteType, Bin> allBins = new()
{
    [WasteType.Glass] =
        new Bin { Type = WasteType.Glass, Image = "a_klaaspakend" },
    [WasteType.Hazardous] =
        new Bin { Type = WasteType.Hazardous, Image = "b_ohklikudjaatmed" },
    [WasteType.Deposit] =
        new Bin { Type = WasteType.Deposit, Image = "c_pandipakend" },
    [WasteType.PaperPackaging] =
        new Bin { Type = WasteType.PaperPackaging, Image = "d_pappjapaberpakend" },
    [WasteType.PMB_Carton] =
        new Bin { Type = WasteType.PMB_Carton, Image = "e_plastmetalljoogikartong" },
    [WasteType.Reusable] =
        new Bin { Type = WasteType.Reusable, Image = "f_ringlusnoud" },
    [WasteType.Mixed] =
        new Bin { Type = WasteType.Mixed, Image = "g_segaolmejaatmed" },
    [WasteType.Bio] =
        new Bin { Type = WasteType.Bio, Image = "h_biojaatmed" },
    [WasteType.ScrapPaper] =
        new Bin { Type = WasteType.ScrapPaper, Image = "h_vanapaber" },
};
```

Joonis 18. Koodilõik - kõigi prügikastide ja kõigi jäätmeliikide loend

Teine sõnastik, **wasteByType**, sisaldab jäätmete nimekirja, mis on rühmitatud jäätmeliikide kaupa. Iga jäätmeliigi jaoks on olemas objektide nimekiri **SortableItem**, kus iga objekt sisaldab objekti nime, viidet kuvamiseks vajalikule pildile ja vastavat jäätmeliiki. Näiteks on klaasi kategoorias märgitud klaaspudel ja klaaspurk, ohtlike jäätmete kategoorias patareid ja lambipirn ning biojäätmete kategoorias õuna- ja banaanikoor.

Mänguvooru raames valib rakendus juhuslikult sõnastikust wasteByType ühe eseme ja määrab selle tüübi, et moodustada vastus sõnastikust allBins. Rakendus moodustab vastuseks konteinerite kogumi: üks õige ja ülejäänud on eksitavad. Pärast vastuse valimist võrdleb rakendus valitud konteineri jäätmetüüpi praeguse eseme jäätmetüübiga, et kindlaks teha, kas vastus on õige.

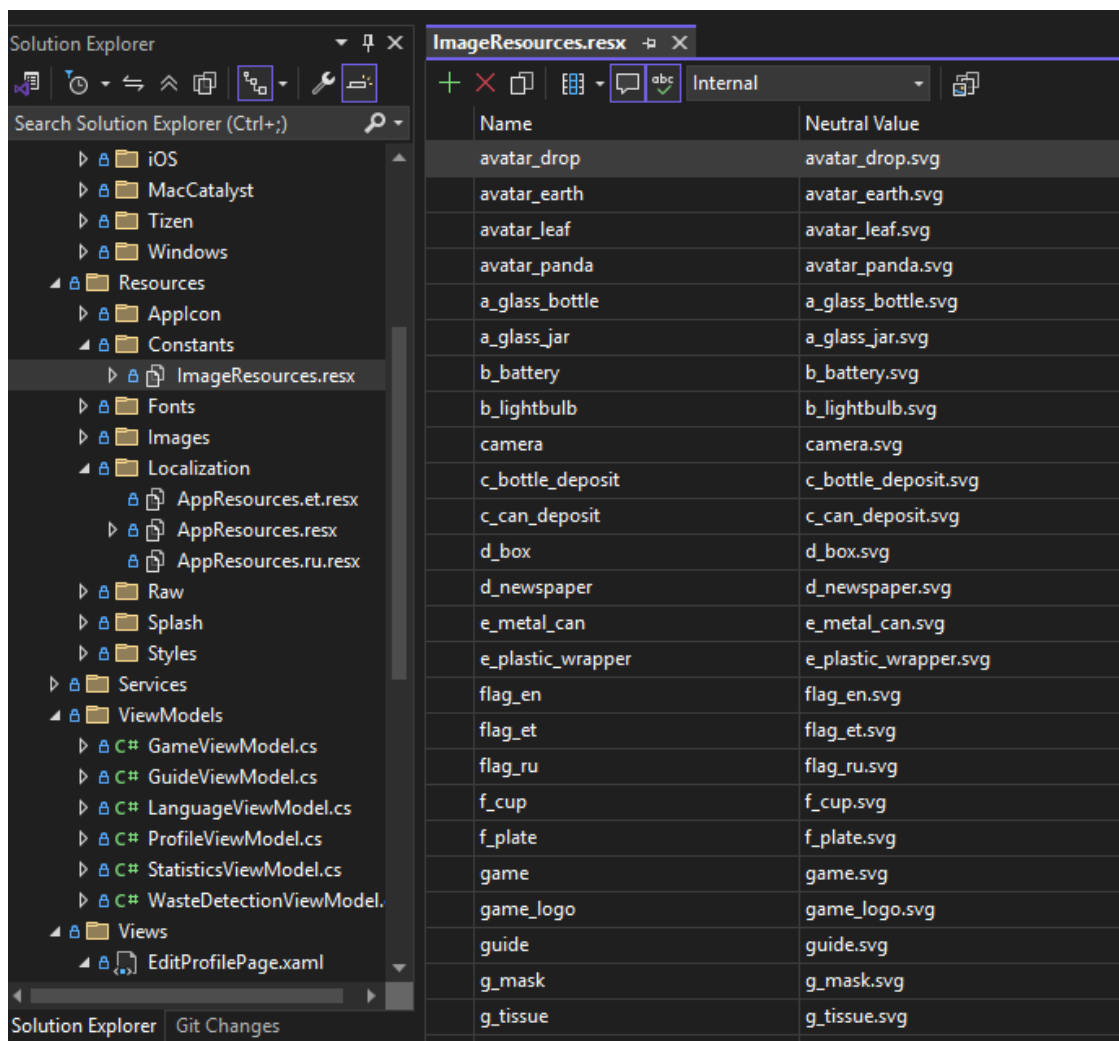
Sõnastike kasutamine lihtsustab mänguloogikat ja muudab selle paindlikumaks: vajaduse korral on võimalik lisada uus jäätmeliik, uus konteiner või uued esemete pildid ilma mänguloogikat oluliselt muutmata. Piisab, kui lisada uus tüüp loendisse WasteType ning vastavad elemendid kogudesse allBins ja wasteByType.

Allpool on esitatud sõnastiku wasteByType realiseering, mis sisaldab rakenduses kasutatavate jäätmeobjektide loendit, grupeerituna jäätmeliikide kaupa:

```
private readonly Dictionary<WasteType, List<SortableItem>> wasteByType = new()
{
    [WasteType.Glass] = new List<SortableItem>
    {
        new SortableItem("GlassBottle", ImageResources.a_glass_bottle, WasteType.Glass),
        new SortableItem("GlassCan", ImageResources.a_glass_jar, WasteType.Glass),
    },
    [WasteType.Hazardous] = new List<SortableItem>
    {
        new SortableItem("Battery", ImageResources.b_battery, WasteType.Hazardous),
        new SortableItem("Bulb", ImageResources.b_lightbulb, WasteType.Hazardous),
    },
    [WasteType.Deposit] = new List<SortableItem>
    {
        new SortableItem("DepositBottle", ImageResources.c_bottle_deposit, WasteType.Deposit),
        new SortableItem("DepositCan", ImageResources.c_can_deposit, WasteType.Deposit),
    },
    [WasteType.PaperPackaging] = new List<SortableItem>
    {
        new SortableItem("Box", ImageResources.d_box, WasteType.PaperPackaging),
        new SortableItem("Newspaper", ImageResources.d_newspaper, WasteType.PaperPackaging),
    },
    [WasteType.PMB_Carton] = new List<SortableItem>
    {
        new SortableItem("FilmWrapping", ImageResources.e_plastic_wrapper, WasteType.PMB_Carton),
        new SortableItem("MetalCan", ImageResources.e_metal_can, WasteType.PMB_Carton),
    },
    [WasteType.Reusable] = new List<SortableItem>
    {
        new SortableItem("ReusableMug", ImageResources.f_cup, WasteType.Reusable),
        new SortableItem("ReusablePlate", ImageResources.f_plate, WasteType.Reusable),
    },
    [WasteType.Mixed] = new List<SortableItem>
    {
        new SortableItem("Napkin", ImageResources.g_tissue, WasteType.Mixed),
        new SortableItem("MedicalMask", ImageResources.g_mask, WasteType.Mixed),
    },
    [WasteType.Bio] = new List<SortableItem>
    {
        new SortableItem("AppleCore", ImageResources.h_apple_core, WasteType.Bio),
        new SortableItem("BananaPeel", ImageResources.h_banana_peel, WasteType.Bio),
    },
    [WasteType.ScrapPaper] = new List<SortableItem>
    {
        new SortableItem("PaperNewspaper", ImageResources.d_newspaper, WasteType.ScrapPaper),
    },
};
```

Joonis 19. Koodilõik - kõikide jäätmeliikide loend, liigitatud tüüpide kaupa

Rakenduses kasutatavate illustratsioonide viited on salvestatud ressursside kausta failis **ImageResources.resx**. Koodis kasutatakse pildi otsese tee asemel vastavat ressursivõtit (muutujat), mis on määratletud selles failis. See hõlbustab koodi edasist muutmist ja muudab selle paindlikumaks: kui on vaja pilti asendada, piisab ressursifailis asuva tee muutmisest ning kõik vajalikud muudatused kajastuvad koodis, kuna võti jääb samaks ja juba töötava rakenduse loogika jätkab korrektset toimimist.



Joonis 20. Rakenduses kasutatavate pildiressursside loend failis ImageResources.resx

Enne mänguvoorude algust algseadistatakse parameetrid. Punktid ja loendurid nullitakse, valitakse esimene ese ja käivitatakse 30-sekundiline taimer. Kui taimer lõpeb (jõuab nullini), lõpeb mäng. See suurendab kasutaja kaasatust ja lisab mängule võistlusmomendi.

Vooru lõppedes salvestatakse tulemused andmebaasi, mis võimaldab neid kasutada statistika kuvamiseks ja kasutaja edusammude arvutamiseks.

Peamised toimingud pärast mängu lõppu:

- lõpppunktide arvutamine
- kogemuspunkte kogumine (XP)
- mängusessiooni salvestamine
- kasutaja profiili uuendamine

See tagab mängutulemuste salvestamise ja võimaldab kasutaja tulemusi edaspidi analüüsida. Mängurežiimi põhiline äriloo­gika on realiseeritud eraldi **GameService** klassis, mis vastutab mängureeglite, punktiarvestuse, vastuste kontrollimise, juhuslike jäätmeobjektide ja konteinerite valimise ning vooru tulemuste arvutamise eest. **GameViewModel** toimib seejuures vahendajana kasutajaliidese ja GameService vahel, hallates kasutaja sisendeid, käskusid, taimerit ning andmete sidumist vaatega.

Allpool on toodud vastuse kontrollimise funktsiooni oluline lõik:

```
// Kontrollib, kas mängija valitud prügikast on õige
public bool CheckAnswer(int tappedIndex)
{
    if (!IsRunning || CurrentItem == null) return false;
    if (tappedIndex < 0 || tappedIndex >= ActiveBins.Count) return false;

    Bin tappedBin = ActiveBins[tappedIndex];
    bool correctAnswer = tappedBin.Type == CurrentItem.Type;

    if (correctAnswer)
    {
        Score += 3;
        Correct++;
    }
    else
    {
        Score -= 3;
        Wrong++;
    }

    App.UserDB.UpdateWasteTypeStat(CurrentItem.Type.ToString(), correctAnswer);

    PickBinsForThisTurn();
    PickNextTrashItem();

    return correctAnswer;
}
```

Joonis 21. Koodilõik - vastuse kontrollimise funktsioon mängurežiimis

MVVM-arhitektuuri kasutamine võimaldas eraldada mänguloogika kasutajaliidestest, mis muudab koodi struktureeritumaks ja hoolduseks lihtsamaks. Mängurežiim ei ole mitte ainult teadmiste kontrollimine, vaid ka vahend nende kinnistamiseks korduvate tegevuste ja tagasiside kaudu.

3.3.2. Andmebaasi haldus

Andmetele süsteemis pääseb ligi eraldatud teenuse DatabaseService kaudu, mis varjab kogu andmebaasiga töötamise loogika. Teenuse käivitamisel ühendub kood SQLite-andmebaasiga, luues vajalikud tabelid:

```
public DatabaseService(string dbPath)
{
    _db = new SQLiteConnection(dbPath);

    _db.CreateTable<UserProfile>();
    _db.CreateTable<GameSessionStat>();
    _db.CreateTable<WasteTypeStat>();
}
```

Joonis 22. Koodilõik - andmebaasi tabelite loomine (DatabaseService)

Esitatud koodilõik näitab, et andmebaas luuakse automaatselt rakenduse esmakordsel käivitamisel ning tabelite struktuur moodustatakse programmiselt. (SQLite, 2024)

Teenus DatabaseService vastutab enamiku kasutaja andmete ja statistikaga seotud toimingute eest.

Kogemuste kogumine (XP):

```
public bool AddXp(int amount)
{
    if (amount <= 0)
        return false;

    var p = GetProfile();

    int before = LevelService.GetLevel(p.Xp);

    p.Xp += amount;
    _db.Update(p);

    int after = LevelService.GetLevel(p.Xp);

    return after > before;
}
```

Joonis 23. Koodilõik - kasutaja kogemuse (XP) uuendamise funktsioon

See funktsioon uuendab kasutaja kogemuspunkte ja kontrollib, kas kasutaja on jõudnud uuele tasemele.

Mängusessiooni salvestamine:

```
// Salvestab mängusessiooni statistika, mida saab hiljem vaadata laiendatud statistikast
public void SaveGameSession(int correctAnswers, int wrongAnswers, int gainedXp)
{
    var session = new GameSessionStat
    {
        PlayedAt = DateTime.Now,
        CorrectAnswers = correctAnswers,
        WrongAnswers = wrongAnswers,
        GainedXp = gainedXp
    };

    _db.Insert(session);
}
```

Joonis 24. Koodilõik - mänguseansi tulemuste salvestamine andmebaasi

Iga mängusessiooni tulemus kantakse eraldi tabelisse ning seda saab hiljem kasutada statistika arvutamiseks ja visualiseerimiseks.

Jäätmeliikide statistika uuendamine:

```
// Uuendab konkreetse jäätmeliigi statistikat, mida saab hiljem vaadata laiendatud statistikast
public void UpdateWasteTypeStat(string wasteType, bool isCorrect)
{
    if (string.IsNullOrEmpty(wasteType))
        return;

    var stat = _db.Table<WasteTypeStat>()
        .FirstOrDefault(x => x.WasteType == wasteType);

    if (stat == null)
    {
        stat = new WasteTypeStat
        {
            WasteType = wasteType,
            CorrectCount = 0,
            WrongCount = 0
        };
        _db.Insert(stat);
    }

    if (isCorrect)
        stat.CorrectCount++;
    else
        stat.WrongCount++;

    _db.Update(stat);
}
```

Joonis 25. Koodilõik - jäätmeliigi statistika uuendamise funktsioon

See funktsioon uuendab iga jäätmeliigi statistikat, mida saab hiljem kasutada diagrammide koostamiseks ja kasutaja tulemuste analüüsimiseks.

Seega on rakenduse suuruse ja koodi keerukuse vähendamiseks kogu andmebaasiga suhtlemine kapseldatud ühte klassi, mis abstrahereib kõik operatsioonid. Käesoleval juhul on SQLite lihtne ja tõhus lahendus, mis ei nõua eraldi serveri seadistamist.

3.3.3. Keelevahetuse loogika

Rakenduses on rakendatud mitmekeelne kasutajaliides, mis muudab selle kasutajatele kättesaadavaks. Keele vahetamine toimub ressursifailidel (AppResources) põhineva lokaliseerimismehhanismi abil.

Rakenduse tekstiressursid on salvestatud eraldi lokaliseerimisfailidesse (.resx), kus iga keele jaoks on ette nähtud oma tõlkevalik. Projektis kasutatakse faile AppResources.resx, AppResources.et.resx ja AppResources.ru.resx, mis sisaldavad kasutajaliidese tekste vastavalt inglise, eesti ja vene keeles.

Iga ressursifail koosneb võtmetest ja neile vastavatest tõlkeväärtustest.

Ressursifailide struktuuri näide on toodud lisas ([LISA D](#)).

Keele vahetamise loogika on rakendatud eraldi teenuse LanguageService kaudu. Kui kasutaja valib keele, toimuvad järgmised toimingud:

- luuakse uus CultureInfo objekt
- määratakse väärtused CurrentCulture ja CurrentUICulture
- rakenduse ressursides uuendatakse kultuuri
- valitud keel salvestatakse seadme sätetesse (Preferences)
- käivitatakse kasutajaliidese uuendamise sündmus

```
public static void ChangeLanguage(string languageCode)
{
    var culture = new CultureInfo(languageCode);
    CultureInfo.DefaultThreadCurrentCulture = culture;
    CultureInfo.DefaultThreadCurrentUICulture = culture;

    AppResources.Culture = culture;

    Preferences.Set("AppLanguage", languageCode);
    LanguageChanged?.Invoke();
}
```

Joonis 26. Koodilõik - keelevahetuse loogika realiseerimine (LanguageService)

Klassis LanguageViewModel luuakse käsud keele valimiseks. Ühe sellise käsu täitmine kutsub välja meetodi, mis vastutab valitud keelele ülemineku eest, ning uuendab valitud keele muutuja:

```
public LanguageViewModel()
{
    SetEnglishCommand = new Command(() => ChangeLanguage("en"));
    SetRussianCommand = new Command(() => ChangeLanguage("ru"));
    SetEstonianCommand = new Command(() => ChangeLanguage("et"));

    LanguageService.LanguageChanged += OnLanguageChanged;
}
```

Joonis 27. Koodilõik - keele valiku käsud ViewModel-is

Keele muutmise järel uuendatakse kõik tekstiomadused INotifyPropertyChanged-mehhanismi kaudu. See võimaldab kasutajaliidest automaatselt uuendada ilma rakendust uuesti käivitamata.

3.3.4. Google Cloud Vision API kasutamine (REST API)

Üks arendatud rakenduse huvitavamaid funktsioone on jäätmete tuvastamine fotode põhjal. Selle funktsiooni realiseerimiseks kasutati teenust **Google Cloud Vision API**, millega suheldakse **REST-arhitektuuri** kaudu. (Google Cloud, 2024)

See režiim võimaldab saata pildi tuvastusteenusele ja saada nimekirja märksõnadest, mis kirjeldavad pildil leitud objekte. Google Cloud Vision API dokumentatsioonis on märgitud, et päring Vision API-le märksõnade tuvastamiseks (Label Detection) saadetakse POST-meetodil aadressile images:annotate ning päringu sisus edastatakse pilt Base64-vormingus

ja analüüsi tüüp LABEL_DETECTION. Samuti on võimalik tulemuste arvu piirata parameetri maxResults abil. Vastuses tagastab API massiivi labelAnnotations, kus iga kirje sisaldab muu hulgas välja description ja score. Välja description tähistab objekti tekstilist märget, samas kui score näitab mudeli kindlustaset vahemikus 0 kuni 1. (Google Cloud, 2024)

Rakenduses „SortIt“ on prügi tuvastamise funktsioon realiseeritud eraldi teenuse CloudVisionAPIService kaudu, mis vastab MVVM-arhitektuurile. Selline lähenemine võimaldas viia suhtluse välise API-ga kasutajaliidestest välja ja paigutada selle teenuskihti. Teenuse konstruktoris loetakse API-võti failist appsettings.json, mille järel moodustatakse päring Google Vision API-le. Allpool on toodud koodilõik, kus toimub teenuse initsialiseerimine ja API-võtme lugemine:

```
// REST API teenus Google Cloud Vision API jaoks
public class CloudVisionAPIService
{
    private readonly HttpClient _httpClient;
    private readonly string _apiKey;
    private readonly string _url;

    public CloudVisionAPIService(HttpClient httpClient)
    {
        _httpClient = httpClient;

        // Loeb API võtme appsettings.json failist
        using var stream = FileSystem.OpenAppPackageFileAsync("appsettings.json").Result;

        var config = new ConfigurationBuilder()
            .AddJsonStream(stream)
            .Build();

        _apiKey = config["GoogleVision:ApiKey"];
        _url = "https://vision.googleapis.com/v1/images:annotate?key=" + _apiKey;
    }
}
```

Joonis 28. Koodilõik - Google Cloud Vision API teenuse initsialiseerimine

Pärast seda, kui kasutaja on objekti pildistanud, peab rakendus pildi teenusele saatmiseks ette valmistama. Pildi võib laadida JSON-kehas üles päringu parameetrina Base64-vormingus, vastavalt Google Cloud Vision API dokumentatsioonile.

Selleks loeb rakendus faili baitide massiivina, teisendab selle Base64-koodiga stringiks ja loob päringuobjekti tüübiga LABEL_DETECTION ning piiranguga maxResults = 5. (Google Cloud, 2024)

Allpool on toodud meetodi DetectObjectAsync põhiline osa, mis moodustab ja saadab selle päringu:

```
public async Task<(string? label, double confidence)> DetectObjectAsync(string imagePath)
{
    if (!File.Exists(imagePath))
        return (null, 0);

    // Loeb pildi byte massiiviks
    byte[] imageBytes = await File.ReadAllBytesAsync(imagePath);

    // Teisendab Base64 formaati
    string base64 = Convert.ToBase64String(imageBytes);

    // Loo me päringu objekti Google Cloud Vision API jaoks
    var requestObject = new
    {
        requests = new[]
        {
            new
            {
                image = new { content = base64 },
                features = new[]
                {
                    new { type = "LABEL_DETECTION", maxResults = 5 }
                }
            }
        }
    };

    // Serialiseerib JSON-iks
    string jsonRequest = JsonSerializer.Serialize(requestObject);

    var content = new StringContent(jsonRequest, Encoding.UTF8, "application/json");

    // Saadab POST päringu
    var response = await _httpClient.PostAsync(_url, content);

    string jsonResponse = await response.Content.ReadAsStringAsync();

    if (!response.IsSuccessStatusCode)
        return (null, 0);

    // Parsib JSON vastuse
    using JsonDocument doc = JsonDocument.Parse(jsonResponse);

    var labels = doc.RootElement
        .GetProperty("responses")[0]
        .GetProperty("labelAnnotations");

    // Võtab esimese tuvastatud objekti
    if (labels.GetArrayLength() > 0)
    {
        string label = labels[0].GetProperty("description").GetString();
        double score = labels[0].GetProperty("score").GetDouble();

        return (label, score);
    }

    return (null, 0);
}
```

Joonis 29. Koodilõik - pildi analüüsi päringu koostamine ja saatmine

See meetod teostab API-ga suhtlemise täieliku tsükli: faili olemasolu kontrollimise, pildi teisendamise, JSON-i serialiseerimise, POST-päringu saatmise ja saadud vastuse analüüsimise. Massiivist labelAnnotations võtab rakendus esimese leitud märgise ja sellega seotud usalduskoefitsiendi.

Tuvastamisfunktsioon on rakenduses realiseeritud klassis WasteDetectionViewModel. Just selles klassis toimub kaameraga töötamine, fotode salvestamine ja tulemuste kuvamine. WasteDetectionViewModel-mudeli objektide loomisel saab kasutaja juurdepääsu fotode loomisele MediaPicker'i abil. Laaditud foto salvestatakse rakenduse vahemällu, mille järel ilmub nupp tuvastusprotsessi käivitamiseks.

Pärast Google Vision API-lt tulemuse saamist tuleb kindlaks määrata, millisesse konteinerisse avastatud jäätmed tuleb visata. Selleks kasutatakse klassi WasteCategoryMapper, milles on määratletud märksõnade sõnastik, mis kajastab erinevate jäätmete tüüpe: plast ja metall, klaas, paber ja papp, biojätmed, ohtlikud jäätmed, elektroonika, riided ja jalatsid, segajätmed ja muud. Meetod MapLabelToContainerKey võimaldab sisestada leitud jäätmete nime väiketähtedega ja teha otsingut märksõnade järgi. Kui otsing ei õnnestu, kasutatakse segajätmete konteinerit.

Lisaks teabele tuvastatud jäätmete kohta kuvab rakendus ka soovitatava konteineri tüübi nende kõrvaldamiseks. See funktsioon suurendab rakenduse praktilist väärtust kasutaja jaoks. Konteinerite kuvamiseks mõeldud tekst määratakse kindlaks objekti AppResources kaudu.

Google Cloud Vision API võimaldab teha üldist pildianalüüsi, mitte aga jäätmeliigi spetsiifilist analüüsi. Teenuse tulemus sõltub suuresti pildi kvaliteedist. Analüüsi võimalikud tulemused võivad sisaldada selliseid sõnu nagu plast, pudel, toit või paber. Saadud tulemuse põhjal määratakse rakenduses kindlaks vastav konteineri tüüp. Sellise lähenemise tõttu loodi klass vastavate jäätmeliikide märgistuste vastavuse jaoks. Tänu sellise klassi rakendamisele on võimalik kasutada universaalset jäätmete tuvastamise teenust konkreetse haridusülesande raames.

Seega võimaldas Google Cloud Vision API kasutamine laiendada rakenduse „SortIt“ funktsionaalsust ja muuta see mitte ainult õppevahendiks, vaid osaliselt ka praktiliseks tööriistaks. Kasutajal on võimalus jäätmetest pilt teha, saata see analüüsimiseks, näha tuvastatud objekti ja otsustada, millisesse konteinerisse see tuleks visata.

See funktsioon sobib loomulikult haridusrakenduse kontseptsiooniga, kuna pakub kasutajale võimalust sorteerimise alaseid teadmisi praktikas rakendada. Lisaks annab see kasutajale mugava vahendi, mis aitab igapäevaelus õigeid otsuseid langetada.

3.3.5. Statistika ja graafikud (Syncfusion)

Väljatöötatud rakenduse peamine eripära on kasutaja statistika kuvamine. See funktsioon võimaldab jälgida õppimise edusamme, analüüsida mängutulemusi ja suurendada kasutaja motivatsiooni. Statistika andmete visualiseerimine rakenduses teostati raamatukogu [Syncfusion](#) abil, mis pakub valmis komponente diagrammide ja graafikute loomiseks rakendustes. (Syncfusion, 2024)

Syncfusioni ametlike dokumentide kohaselt toetab see raamatukogu laia valikut diagrammiliike (sambad, jooned, ringdiagrammid jne) ning pakub paindlikke võimalusi andmete kuvamiseks.

„SortIt“ rakenduses on statistika töötlemise loogika rakendatud klassis `StatisticsViewModel` täielikus kooskõlas MVVM-arhitektuuriga. See `ViewModel` vastutab andmete hankimise eest andmebaasist ja nende töötlemise eest, et moodustada kuvatavad andmed.

Statistikaandmed pärinevad kohalikust SQLite-andmebaasist `DatabaseService`'i kaudu. Täpsemalt kasutatakse järgmisi andmeid: õigete vastuste koguarv, valede vastuste arv, vastuste koguarv, kogutud kogemuspunktid (XP) ja mängusessioonide ajalugu.

Töödeldud andmed vormistatakse formaadis, mis on vajalik nende edasiseks kuvamiseks graafikutel. Selleks luuakse spetsiaalsed kogumid (`OverallPerformanceSeries`) ja täidetakse vajalikud „märged“, „väärtused“ paarid. (Syncfusion, 2024)

Andmete kuvamiseks kasutatakse Syncfusion-raamatukogu, mis on integreeritud XAML-liidesega. Andmete muutmine `ViewModel`'is põhjustab diagrammi automaatse uuendamise.

Rakenduses on realiseeritud järgmised visualiseerimisviisid:

- õigete ja valede vastuste jaotus
- täpsus
- kasutaja üldine edu

- kasutaja staatus
- kasutaja tase
- jäätmeliikide statistika
- kogemuspunkte (XP) mängusessioonide kaupa (kuude või päevade kaupa)

Statistika on õppeprotsessi oluline osa: see annab kasutajale teavet tema edusammude kohta, võimaldab analüüsida vigu ja jälgida vajalike oskuste arengut. Erinevate diagrammide ja graafikute kasutamine muudab õppeprotsessi selgemaks ja huvitavamaks. Seega õnnestus käesoleva osa rakendamisel luua „SortIt“ rakenduses Syncfusion-raamatukogu abil võimalikult intuitiivne ja ligipääsetav statistika kuvamise süsteem.

4. TESTIMINE

Testimine on tarkvaraarenduse oluline etapp, kuna see võimaldab veenduda rakenduse funktsioonide korrektse töö, avastada vigu ja kontrollida, kas programm vastab nõuetele. Selle projekti raames viidi läbi automaatne testimine ühikutestide abil ning manuaalne testimine.

4.1. Automaatne testimine (Unit testid)

Testimiseks kasutati eraldi projektis SortItTests loodud ühikteste. Testid käivitati Visual Studio Test Exploreri kaudu. Ühiktestimine võimaldab kontrollida programmi üksikuid funktsioone ja meetodeid kasutajaliidesest sõltumatult.

4.1.1. Kasutajaprofiili testimine

Klassis UserProfileTests testiti kasutajaprofiili loogikat.

Kontrolliti:

- profiili loomine vaikimisi väärtustega
- kasutajakogemuse uuendamine

```
// Kontrollib kasutajaprofiili vaikeväärtusi uue profiili loomisel
[Fact]
public void UserProfile_ShouldHaveDefaultValues()
{
    // Arrange
    UserProfile profile = new UserProfile();

    // Assert
    Assert.Equal("Eco Hero", profile.Name);
    Assert.Equal("avatar_leaf.svg", profile.Avatar);
    Assert.Equal(0, profile.Xp);
    Assert.Equal(0, profile.TotalCorrect);
    Assert.Equal(0, profile.TotalWrong);
}
```

Joonis 30. Koodilõik - kasutajaprofiili testimine

4.1.2. Mängurežiimi loogika testimine

Klassis `GameLogicTests` kontrolliti punktide andmise loogikat ning õigete ja valede vastuste arvestamist. Testiti järgmisi stsenaariume:

- õige vastus suurendab õigete vastuste arvu
- õige vastus suurendab kogemuspunkte (XP)
- teatud kogemuste taseme saavutamisel muutub tase
- vale vastus ei vähenda kogemuspunkte
- vale vastus suurendab vigade arvu

```
// Kontrollib, et õige vastus suurendab kasutaja XP-d
[Fact]
public void CorrectAnswer_ShouldIncreaseXp()
{
    // Arrange
    UserProfile profile = new UserProfile();
    int xpBefore = profile.Xp;

    // Act
    profile.Xp += 10;

    // Assert
    Assert.True(profile.Xp > xpBefore);
}

// Kontrollib, et õige vastus suurendab õigete vastuste arvu
[Fact]
public void CorrectAnswer_ShouldIncreaseCorrectCount()
{
    // Arrange
    UserProfile profile = new UserProfile();
    int correctBefore = profile.TotalCorrect;

    // Act
    profile.TotalCorrect++;

    // Assert
    Assert.Equal(correctBefore + 1, profile.TotalCorrect);
}

// Kontrollib, et XP = 100 korral muutub kasutaja tase 2-ks
[Fact]
public void Level_ShouldChangeTo2_WhenXpIs100()
{
    // Arrange
    int xp = 100;

    // Act
    int level = LevelService.GetLevel(xp);

    // Assert
    Assert.Equal(2, level);
}
```

Joonis 31. Koodilõik - mänguloogika testimise näited

4.1.3. Tasemesüsteemi testimine

Klassis LevelServiceTests testiti kasutaja tasemete ja edusammude süsteemi.\

Kontrolliti järgmisi funktsioone:

- taseme määramine kogemuse põhjal
- edusammude arvutamine järgmise tasemeni
- pildi tähtsuse määramine
- järgmisele tasemele jõudmiseks vajaliku kogemuse arvutamine

```
// Kontrollib, et kui XP = 0, siis tase on 1
[Fact]
public void GetLevel_ShouldReturn1_WhenXpIs0()
{
    // Arrange
    int xp = 0;

    // Act
    int result = LevelService.GetLevel(xp);

    // Assert
    Assert.Equal(1, result);
}

// Kontrollib, et kui XP = 100, siis tase on 2
[Fact]
public void GetLevel_ShouldReturn2_WhenXpIs100()
{
    // Arrange
    int xp = 100;

    // Act
    int result = LevelService.GetLevel(xp);

    // Assert
    Assert.Equal(2, result);
}

// Kontrollib, et taseme 1 jaoks tagastatakse õige pilt
[Fact]
public void GetRankImage_ShouldReturnSeedling_WhenLevelIs1()
{
    // Arrange
    int level = 1;

    // Act
    string result = LevelService.GetRankImage(level);

    // Assert
    Assert.Equal("plant_rank0_seedling.svg", result);
}
```

Joonis 32. Koodilõik - tasemesüsteemi testimise näited

4.1.4. Jäätmekategooriate vastendamise testimine

Klassis WasteCategoryMapperTests testiti jäätmekategooria määramise loogikat, tuginedes Google Cloud Vision API-st saadud tuvastatud siltidele.

Testiti järgmisi stsenaariume:

- märgise ja konteineri võrdlemine, arvestamata suur- ja väiketähti
- BioWaste kategooria määratlemine
- tühja märkide loendi töötlemine
- nullväärtuste töötlemine
- tundmatute märgiste töötlemine

```
// Kontrollib, et "banana peel" liigitatakse BioWasteContainer kategooriasse
[Fact]
public void MapLabelToContainerKey_ShouldReturnBioWaste_WhenLabelIsBananaPeel()
{
    // Arrange
    string label = "banana peel";

    // Act
    string result = WasteCategoryMapper.MapLabelToContainerKey(label);

    // Assert
    Assert.Equal("BioWasteContainer", result);
}

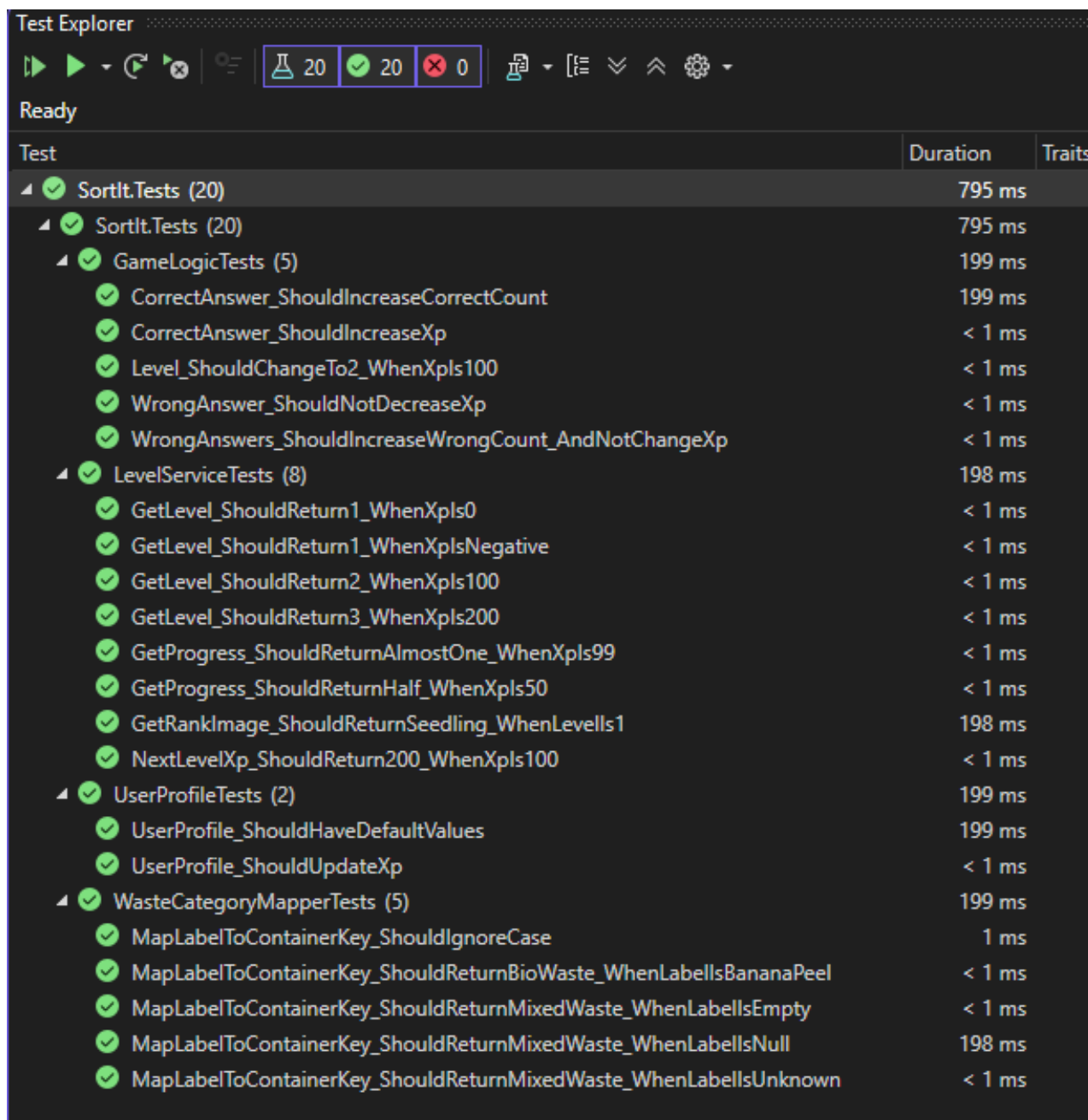
// Kontrollib, et tundmatu ese liigitatakse MixedWasteContainer kategooriasse
[Fact]
public void MapLabelToContainerKey_ShouldReturnMixedWaste_WhenLabelIsUnknown()
{
    // Arrange
    string label = "unknown item";

    // Act
    string result = WasteCategoryMapper.MapLabelToContainerKey(label);

    // Assert
    Assert.Equal("MixedWasteContainer", result);
}
```

Joonis 33. Koodilõik - jäätmekategooriate määramise loogika testimine

Funktsionaalsete testide käigus viidi läbi 20 üksiktesti, millega kontrolliti rakenduse põhilisi algoritme. Kõik testid läbisid edukalt ja veatult, mis kinnitab järgmiste osade korrektset toimimist: mänguloogika, tasemesüsteem, kasutajaprofiil, jäätmekategooriate sobitamise algoritm.



Test	Duration	Traits
SortIt.Tests (20)	795 ms	
SortIt.Tests (20)	795 ms	
GameLogicTests (5)	199 ms	
CorrectAnswer_ShouldIncreaseCorrectCount	199 ms	
CorrectAnswer_ShouldIncreaseXp	< 1 ms	
Level_ShouldChangeTo2_WhenXpls100	< 1 ms	
WrongAnswer_ShouldNotDecreaseXp	< 1 ms	
WrongAnswers_ShouldIncreaseWrongCount_AndNotChangeXp	< 1 ms	
LevelServiceTests (8)	198 ms	
GetLevel_ShouldReturn1_WhenXpls0	< 1 ms	
GetLevel_ShouldReturn1_WhenXplsNegative	< 1 ms	
GetLevel_ShouldReturn2_WhenXpls100	< 1 ms	
GetLevel_ShouldReturn3_WhenXpls200	< 1 ms	
GetProgress_ShouldReturnAlmostOne_WhenXpls99	< 1 ms	
GetProgress_ShouldReturnHalf_WhenXpls50	< 1 ms	
GetRankImage_ShouldReturnSeedling_WhenLevels1	198 ms	
NextLevelXp_ShouldReturn200_WhenXpls100	< 1 ms	
UserProfileTests (2)	199 ms	
UserProfile_ShouldHaveDefaultValues	199 ms	
UserProfile_ShouldUpdateXp	< 1 ms	
WasteCategoryMapperTests (5)	199 ms	
MapLabelToContainerKey_ShouldIgnoreCase	1 ms	
MapLabelToContainerKey_ShouldReturnBioWaste_WhenLabellsBananaPeel	< 1 ms	
MapLabelToContainerKey_ShouldReturnMixedWaste_WhenLabellsEmpty	< 1 ms	
MapLabelToContainerKey_ShouldReturnMixedWaste_WhenLabellsNull	198 ms	
MapLabelToContainerKey_ShouldReturnMixedWaste_WhenLabellsUnknown	< 1 ms	

Joonis 34. Koodilõik - testide käivitamise tulemused (Test Explorer)

4.2. Manuaalne testimine

Käsitsi testimist viidi läbi kogu rakenduse arendamise vältel. Selle etapi eesmärk oli kontrollida kasutajaliidese korrektset toimimist ja rakenduse kasutusmugavust ning leida võimalikke vigu, mida automaatse testimise käigus ei avastatud.

Testimine viidi läbi mobiilseadmel ja Visual Studio arenduskeskkonda emuleerivas keskkonnas. Erilist tähelepanu pöörati rakenduse töökindlusele, ekraanide vahelise navigeerimise sujuvusele ja andmete õigele kuvamisele.

Käsitsi testimise käigus kontrolliti järgmisi rakenduse kasutusstsenaariume:

- Õpperežiim (Learning Mode)
 - Jäätmete alase teabe õige kuvamine
 - elementide vahelise ülemineku kontrollimine
 - jäätmekategooriate vastavuse kontroll
- Mängurežiim (Game Mode)
 - jäätmete ja konteinerite õige kuvamine
 - konteineri valiku kontrollimine
 - punktide lisamise ja mahaarvamise kontrollimine
 - ajastiku töö kontrollimine
 - järgmise objekti juurde liikumise kontrollimine
 - mängu lõppedes tulemuse kuvamise kontrollimine
- Prügi tuvastamine (Waste Detection)
 - kaamera töökontroll
 - pildi laadimise kontroll
 - pildi saatmise kontrollimine Google Cloud Vision API-s
 - tuvastustulemuse õige kuvamise kontrollimine
 - veatöötuse kontroll (kui objekti ei tunnistata)
- Statistika (Statistics)
 - graafikute kuvamise kontrollimine
 - andmete õigsuse kontrollimine (punktid, vastused)
 - statistika uuendamise kontrollimine pärast mängu
- Kasutaja profiil (Profile)
 - nime ja avatari kuvamise kontrollimine

- avatari muutmise kontrollimine
- taseme ja kogemuspunkte (XP) kontrollimine
- auastme ja edusammude kuvamise kontrollimine
- Seaded (Settings)
 - keele valiku kontroll (eesti, inglise, vene)
 - teema vahetuse kontroll (heleda/tumeda)
 - heli ja vibratsiooni kontrollimine

Käsitsi testimise käigus avastati ja parandati järgmised probleemid: statistika ebaõige uuendamine mängu lõppedes, teksti kuvamisvead keele vahetamisel, mõnede graafikute ebaõige kuvamine.

Käsitsi testimine näitas, et rakendus on stabiilne, toimiv ja kasutajasõbralik. Kõik põhilised funktsioonid töötavad korrektselt, vead kõrvaldati arendamise käigus ning kasutajaliides vastab seatud nõuetele.

5. KASUTUSJUHE

Käesolev juhend on mõeldud kasutajatele ning selgitab rakenduse põhilisi funktsioone ja selle kasutamist.

Allpool on esitatud videojuhend, milles tutvustatakse rakenduse põhilisi funktsioone ja selle kasutamist (pildile vajutades).



Video 1. Mobiilirakenduse kasutusjuhend

5.1. Profiili seadistamine

Profiili osas saab kasutaja: sisestada või muuta nime, valida avatari, vaadata taset ja edusamme ning tutvuda statistikaga. Muudatuste salvestamiseks tuleb vajutada nuppu **Salvesta**.

5.2. Mängurežiimi kasutamine

Mängurežiim on mõeldud jäätmete sorteerimise õppimiseks.

Kasutamishend:

- Vajuta nuppu **Start**
- Ekraanile ilmub jäätmete pilt ja mitu konteinerit
- Valida jäätmeliigile vastav konteiner
- Saada tulemus:
 - õige vastus - antakse punkte
 - vale vastus - registreeritakse viga ja arvatakse punkte maha

Pärast vooru lõppu kuvatakse tulemus.

5.3. Jäätmete tuvastamine

Tuvastamisfunktsioon võimaldab pildi põhjal kindlaks teha jäätmete tüübi.

Kasutamishend:

- Mine jaotisesse **Tuvasta**
- Tee foto, vajutades nuppu **Tee foto**
- Vajuta nuppu **Tuvasta prügi** ja oota analüüsi tulemusi
- Tutvuda tulemuste ja sorteerimissoovitustega

5.4. Statistika vaatamine

Kui kasutaja läheb jaotisesse **Profiil**, saab ta vaadata põhilisi statistilisi andmeid, sealhulgas kogutud kogemuspunkte (XP), praegust taset, auastet ning õigete ja valede vastuste arvu.

Täpsema teabe saamiseks tuleb klõpsata nuppu **Rohkem**, mille järel avaneb jaotis **Statistika**.

Selles jaotises kuvatakse kolm graafikut, mis võimaldavad analüüsida kasutaja tulemusi:

- teist ja kolmandat graafikut saab horisontaalselt kerida
- teisel graafikul kuvatakse veergudele klõpsates teavet vastuste arvu kohta
- kolmas graafik näitab kasutaja aktiivsust kogutud kogemuste näol

Kolmandas graafikus on võimalik valida kuvatavat ajavahemikku:

- kui valida **Kõik kuud**, kuvatakse statistika kuude kaupa
- konkreetse kuu valimisel kuvatakse andmed päevade kaupa
- aasta valimiseks kasutatakse nuppu **Vali aasta**

5.5. Seadete kasutamine

Seadete osas on saadaval järgmised funktsioonid:

- keelevalik (inglise, eesti, vene)
- kujunduse teema valik (heleda/tumeda)
- heli sisse- või väljalülitamine
- nupp **Välja** rakendusest väljumiseks
- nupp **Lähenda profiili** profiili tühistamiseks

Muudatused hakkavad kehtima kohe pärast valiku tegemist.

6. EDASIARENDAmise VÕIMALUSED

Lisaks „SortIt“ rakenduses juba olemasolevatele põhifunktsioonidele on mitmeid võimalusi selle edasiseks arendamiseks ja laiendamiseks.

Üks peamisi prioriteete järgmistes arendusetappides on üleminek serveripõhisele andmebaasile, mis võimaldab rakendada kasutajate registreerimist ja autentimist, andmete sünkroniseerimist pilves ning rakenduse samaaegset kasutamist mitmel seadmel. Selle alusel oleks võimalik rakendada mingit võistlussüsteemi saavutuste, edetabelite ja mõningate katsetustega. See suurendab kasutajate motivatsiooni rakendust edasi kasutada.

Teine võimalus on lisada kaart, millel on märgitud jäätmete sorteerimiskonteinerite asukohad, mis võimaldab kasutajatel leida lähimaid jäätmekogumispunkte. Näiteks võiks kasutaja, kasutades otsingukriteeriumina teatud jäätmeliiki, leida Eestis selle jäätmeliigi jaoks lähima konteineri. Selleks võib kasutada välist teenust, nagu [Kuhuvia](#), mis kogub teavet jäätmekogumispunktide kohta. Pärast vajaliku API-võtme saamist saab selle teabe rakendusse integreerida ja kuvada konteinerite asukohti kaardil reaalsajas.

Samuti on võimalikud järgmised arengustsenaariumid:

- jäätmete tuvastamise täpsuse suurendamine
- uute mängurežiimide lisamine
- statistika- ja analüüsisüsteemi väljatöötamine
- kasutajale kohandatud soovitude pakkumine

Seega on rakendusel suur arengupotentsiaal ning seda on võimalik laiendada nii tehniliste täiustuste kui ka funktsionaalsuse laiendamise kaudu.

6.1. Piirangud ja puudused

Rakenduse arendamise käigus selgusid järgmised piirangud.

Esiteks sõltub jäätmete tuvastamise funktsioon Google Cloud Vision API kasutamisest, mis eeldab internetiühendust. Kui internetiühendust pole, ei ole see funktsioon kättesaadav.

Teiseks ei ole kasutatav API jäätmete tuvastamiseks mõeldud spetsiaalne teenus, mistõttu jäätmete tuvastamise tulemused ei pruugi alati olla täpsed ning sõltuvad näiteks pildi kvaliteedist, valgustingimustest, asendist ja pildistamisnurgast.

Lisaks kasutab rakendus kohalikku SQLite-andmebaasi, mistõttu salvestatakse andmed ainult kasutaja seadmesse ja neid ei saa mitme seadme vahel sünkroniseerida.

KOKKUVÕTE

Käesoleva lõputöö eesmärk on töötada välja mobiilirakendus, mis toetab õppimis- ja mänguprotsessi jäätmete sorteerimise reeglite omandamisel, kasutades programmeerimiskeelt C# ja platvormi .NET MAUI.

Käesoleva töö käigus viidi läbi valdkonna analüüs, mis võimaldas kindlaks teha jäätmete sorteerimise probleemi aktuaalsuse ning kasutajate seas valitseva ebapiisava teadlikkuse. Selle probleemi lahenduseks pakuti välja mobiilirakenduse loomine.

Analüüsiti rakenduse, selle kasutajaliidese ja andmebaasi võimalikke arhitektuure. Selles projektis valiti MVVM-arhitektuur, mis võimaldas selgelt eraldada kasutajaliidese, loogika ja andmetöötluse, mille tulemusena sai kood struktureeritud ja sobivaks uute funktsioonide edasiseks arendamiseks.

Rakenduses on realiseeritud järgmised funktsioonid:

- õpperežiim
- mängurežiim
- jäätmete tuvastamine pildi põhjal Google Cloud Vision API abil
- statistika, andmete visualiseerimine
- punktisüsteem, tasemed
- mitmekeelne kasutajaliides

Rakenduses kohalike andmete salvestamiseks rakendati SQLite-andmebaasi. Projekti arendamisel kasutati selliseid tööriistu nagu Visual Studio, GitHub, Sourcetree ja Jira. Samuti viidi projekti raames läbi rakenduse automaatne testimine funktsionaalsuse korrektsuse kontrollimiseks ühikutestide abil, samuti käsitsi testimine.

Lõputöö tulemuseks on toimiv mobiilirakendus „SortIt“. Arendatud rakendus on praktilise väärtusega ja seda saab kasutada jäätmete sorteerimise teema õpetamiseks. Jäätmete sorteerija sisaldab õppimis- ja mänguelemente, mis suurendavad õppimismotivatsiooni.

Seega on lõputöö eesmärk saavutatud ja väljatöötatud rakendus vastab kõigile käesoleva töö raames seatud nõuetele.

KASUTATUD KIRJANDUS

- Atlassian. (2024). *Agile epics: definition, examples, and templates*. Allikas: <https://www.atlassian.com/agile/project-management/epics>
- Atlassian. (2025). *Agile Best Practices and Tutorials*. Allikas: <https://www.atlassian.com/agile>
- Atlassian. (2026). *Jira software*. Allikas: <https://www.atlassian.com/software/jira>
- AWS. (2024). *What is Unit Testing? - Unit Testing Explained - AWS*. Allikas: <https://aws.amazon.com/what-is/unit-testing/#:~:text=A%20unit%20test%20is%20a,developer's%20theoretical%20logic%20behind%20it.>
- Banwo, S. (14. Jaanuar 2025. a.). *PNG vs SVG: Which is Best for iOS?* Allikas: https://medium.com/@seun_banwo/png-vs-svg-which-is-best-for-ios-101904730dc2
- Calisir, A. (8. Detsember 2024. a.). *1.5 Years of .NET MAUI: Why I Love It and What Concerns Me*. Allikas: <https://medium.com/@alpay18070005016/1-5-years-of-net-maui-why-i-love-it-and-what-concerns-me-bd9be9452d3e>
- DeepL. (2026). *DeepL Translator Teksti tõlkimiseks*. Allikas: <https://www.deepl.com/translator>
- DisainVeeb. (10. September 2022. a.). *Mis on API? - Disainveeb*. Allikas: <https://disainveeb.ee/blogi/mis-on-api/>
- Erickson, J. (2022). *What is JSON?* Allikas: <https://www.oracle.com/database/what-is-json/>
- Eurostat. (8. Veebruar 2024. a.). *249 kg per person recycled in the EU*. Allikas: <https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20240208-2>
- Git. (2024). *Git*. Allikas: <https://git-scm.com/>
- Google. (2026). *Google Translate - Teksti tõlkimiseks*. Retrieved from <https://translate.google.com/>
- Google Cloud. (2024). *Cloud Vision API documentation*. Allikas: <https://docs.cloud.google.com/vision/docs>
- Google Cloud. (2024). *Label detection*. Allikas: https://docs.cloud.google.com/vision/docs/labels#vision_label_detection-additional-langs
- IBM. (10. Jaanuar 2025. a.). *User experience (UX)*. Allikas: <https://www.ibm.com/think/topics/user-experience>

- Learn Microsoft. (7. Mai 2025. a.). *XAML language overview - WPF*. Allikas: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/>
- Liigitikogumine. (2025). Allikas: Recycling pictograms system: <https://liigitikogumine.ee/>
- Linder, N., Giusti, M., Samuelsson, K., & Barthel, S. (14. September 2021. a.). *Pro-environmental habits: An underexplored research agenda in sustainability science*. Allikas: Springer Nature: <https://link.springer.com/article/10.1007/s13280-021-01619-6>
- Majuria, J., Koivisto, J., & Hamari, J. (21. Mai 2018. a.). *Gamification of education and learning: A review of empirical*. Allikas: Tampere University: https://trepo.tuni.fi/bitstream/handle/10024/104598/gamification_of_education_2018.pdf
- MDN Contributors. (3. August 2019. a.). *HTTP*. Allikas: <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- MDN Contributors. (10. Detsember 2025. a.). *Base64 - MDN Web Docs Glossary: Definitions of Web-related terms | MDN*. Allikas: <https://developer.mozilla.org/en-US/docs/Glossary/Base64>
- Microsoft. (2023). *.NET MAUI documentation*. Allikas: Microsoft: <https://learn.microsoft.com/en-us/dotnet/maui/?view=net-maui-10.0>
- Microsoft Learn. (2. April 2025. a.). *What is .NET MAUI?* Allikas: <https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui?view=net-maui-10.0>
- Minelgaitė, A., & Liobikienė, G. (1. Juuli 2019. a.). *The problem of not waste sorting behaviour, comparison of waste sorters and non-sorters in European Union: Cross-cultural analysis*. Allikas: ScienceDirect: <https://www.sciencedirect.com/science/article/abs/pii/S0048969719313403>
- Radigan, D. (2024). *What is kanban?* Allikas: <https://www.atlassian.com/agile/kanban>
- Sommerville, I. (2016). *Software engineering (10th ed.)*. Pearson.
- SQLite. (2019). *SQLite Home Page*. Allikas: <https://sqlite.org/>
- SQLite. (2024). *SQLite documentation*. Allikas: <https://www.sqlite.org/docs.html>
- Statista. (Juuli 2025. a.). *Smartphone usage worldwide*. Allikas: <https://www.statista.com/statistics/330695/number-of-smartphone-users-worldwide/>
- Syncfusion. (2024). *.NET MAUI charts documentation*. Allikas: <https://help.syncfusion.com/maui/cartesian-charts/overview>
- w3schools. (2022). *C# Tutorial (C Sharp)*. Allikas: <https://www.w3schools.com/cs/index.php>

- Wang, X., & Zhou, D. (10. Jaanuar 2021. a.). *Spatial agglomeration and driving factors of environmental pollution: A spatial analysis*. Allikas: ScienceDirect:
<https://www.sciencedirect.com/science/article/abs/pii/S0959652620338841>
- Wikipedia. (13. April 2010. a.). *Kasutajaliides*. Allikas:
<https://et.wikipedia.org/wiki/Kasutajaliides>
- Wikipedia. (12. Mai 2019. a.). *GitHub*. Allikas: <https://en.wikipedia.org/wiki/GitHub>
- Wikipedia. (22. Juuli 2019. a.). *Jira (software)*. Allikas:
[https://en.wikipedia.org/wiki/Jira_\(software\)](https://en.wikipedia.org/wiki/Jira_(software))
- Wikipedia. (1. Aprill 2019. a.). *Scalable Vector Graphics*. Allikas:
<https://en.wikipedia.org/wiki/SVG>
- Wikipedia. (11. 10 2020. a.). *Model–view–viewmodel*. Allikas:
<https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93viewmodel>
- Wikipedia. (20. Veebruar 2025. a.). *Experience point*. Allikas:
https://en.wikipedia.org/wiki/Experience_point

LISA A. Projekti planeerimine (Jira)

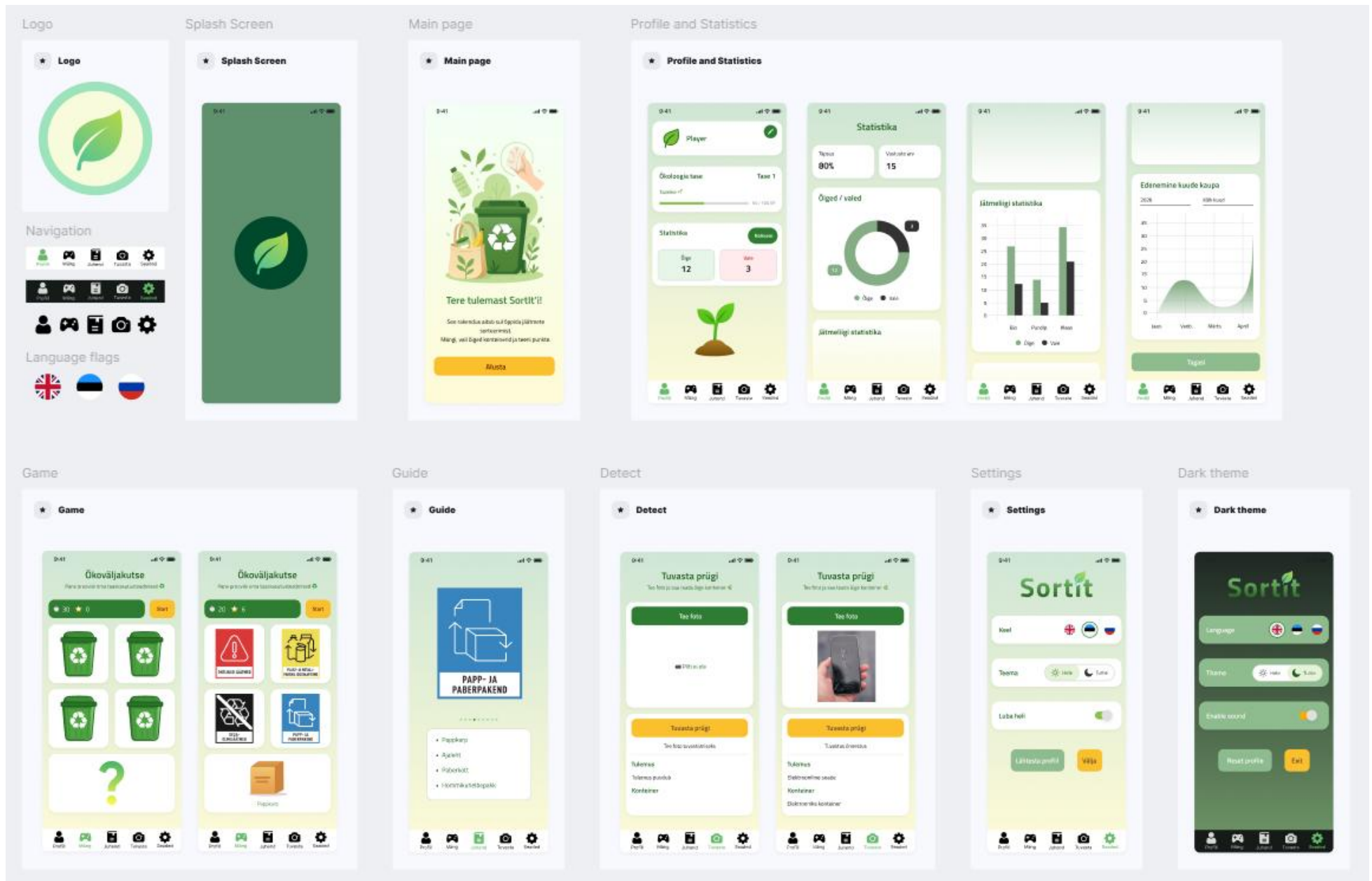
Work		October - December '25	January - March	April -
Releases				
SI-1 EPIC 1 - Projekti algatamine ja tehniline ettevalmist... DONE				
SI-2 Projekti idee ja eesmärgi sõnastamine DONE MARIA SM...				
SI-3 Arendusvahendite ja tehnoloogiate valik DONE MARIA SM...				
SI-5 GitHub repositooriumi loomi... DONE MARIA SM...				
SI-10 Figma stiiliraamatu tegemine TO DO MARIA SM...				
SI-9 Jira töövoo ja ülesannete struktuuri loomi... IN PROGRESS MARIA SM...				
SI-11 EPIC 2 - Kasutajaliidese arendamine DONE				
SI-12 Avalike loomine DONE MARIA SM...				
SI-14 Kasutajaprofiili lehe loomi... DONE MARIA SM...				
SI-18 Piltide ja fontide lisami... IN PROGRESS MARIA SM...				
SI-13 Juhendilehe loomi... DONE MARIA SM...				
SI-16 Seadete lehe loomi... DONE MARIA SM...				
SI-17 Rakenduse navigeerimise seadistami... DONE MARIA SM...				
SI-20 Rakenduse logo ja splash screeni lisamine DONE MARIA SM...				
SI-22 Kasutajaliidese stiilide täiustami... DONE MARIA SM...				
SI-23 EPIC 3 - Andmebaas ja andmehald... DONE				
SI-32 Andmebaasi süsteemi loomine DONE MARIA SM...				
SI-37 Kasutajaprofiili funktsionaalsuse loomi... DONE MARIA SM...				
SI-40 Mangutulemuste salvestami... DONE MARIA SM...				
SI-93 Statistika andmete salvestamine graafiku... DONE MARIA SM...				
SI-53 EPIC 4 - Mänguloogika arendamine DONE				
SI-44 Mangu andmemudelite loomi... DONE MARIA SM...				
SI-47 Mangu põhiloogika realiseerimine DONE MARIA SM...				
SI-50 Mangu kasutajaliidese loomi... DONE MARIA SM...				
SI-55 EPIC 5 - Rakenduse lisafunktsioon... DONE				
SI-24 Lokaliseerimissüsteemi loomine DONE MARIA SM...				
SI-28 Teemade süsteemi realiseerimine DONE MARIA SM...				
SI-57 Heli ja vibratsiooni süsteemi realiseerimine DONE MARIA SM...				
SI-86 Jäätmeliigi ja konteineri automaatse määramise loogika loomine DONE MARIA SM...				
SI-63 EPIC 6 - Testimine ja veaparand...				
SI-73 Vigade parandamine TO DO MARIA SM...				
SI-21 Rakenduse põhifunktsioonide manuaalne testimine IN PROGRESS MARIA SM...				
SI-68 Rakenduse Unit-testide kirjutamine ja käivitamine DONE MARIA SM...				
SI-74 EPIC 7 - Dokumentatsioon				
SI-75 README faili koostami... DONE MARIA SM...				
SI-76 Lõputöö kirjutami... IN PROGRESS MARIA SM...				

LISA B. Versioonihaldus (Sourcetree)

The screenshot shows the Sourcetree application interface. The top menu bar includes File, Edit, View, Repository, Actions, Tools, and Help. Below the menu is a toolbar with icons for Commit, Pull, Push, Fetch, Branch, Merge, Stash, Discard, and Tag. On the right side of the toolbar are icons for Git-flow, Remote, Terminal, Explorer, and Settings. The left sidebar contains a 'WORKSPACE' section with 'File Status', 'History' (selected), and 'Search'. Below this are sections for 'BRANCHES' (showing 'main'), 'TAGS', 'REMOTES', and 'STASHES'. The main area displays a commit history table with columns: Graph, Description, Date, Author, and Commit. The table shows a list of commits, with the top entry being 'Uncommitted changes' and the bottom entry being 'Initial commit'.

Graph	Description	Date	Author	Commit
○	Uncommitted changes	27 apr 2026 4:53	*	*
○	SI-22 Kasutajaliidese stiilide täiustamine	24 apr 2026 3:27	mariasmolina <mariaa.smolina@gmail.com>	9111c35
○	SI-19 Piltide lisamine SVG-formaadis ja ImageResources faili loomine	24 apr 2026 3:15	mariasmolina <mariaa.smolina@gmail.com>	ae4c306
○	SI-73 Vigade parandamine (Lokaliseerimine, Seaded ja Statistika)	7 apr 2026 21:48	mariasmolina <mariaa.smolina@gmail.com>	927764c
○	SI-68 Rakenduse Unit-testide kirjutamine ja käivitamine	6 apr 2026 12:43	mariasmolina <mariaa.smolina@gmail.com>	a8df679
○	SI-93 Statistika andmete salvestamine graafikuna	5 apr 2026 20:03	mariasmolina <mariaa.smolina@gmail.com>	51857a7
○	SI-92 Prügi tuvastamine - hele ja tume teema tugi lisatud	2 apr 2026 16:14	mariasmolina <mariaa.smolina@gmail.com>	5daa084
○	SI-86 Jäätmeliigi ja konteineri automaatse määramise loogika loomine	2 apr 2026 12:48	mariasmolina <mariaa.smolina@gmail.com>	675792a
○	README done	2 ноя 2025 16:45	Maria Smolina <mariaa.smolina@gmail.com>	970f34c
○	README done	2 ноя 2025 16:44	Maria Smolina <mariaa.smolina@gmail.com>	6c6ab7d
○	fix bugs and Test button added	2 ноя 2025 16:04	mariasmolina <mariaa.smolina@gmail.com>	ced8872
○	GamePage, GameViewModel, RoundResult and WasteModels	2 ноя 2025 14:28	mariasmolina <mariaa.smolina@gmail.com>	664b076
○	Localization added more words, Styles modified	1 ноя 2025 22:09	mariasmolina <mariaa.smolina@gmail.com>	27b0cd2
○	Theme added, GlobalStyles	1 ноя 2025 22:08	mariasmolina <mariaa.smolina@gmail.com>	64480c2
○	UserProfile Model	31 окт 2025 13:45	mariasmolina <mariaa.smolina@gmail.com>	31892cb
○	GuidePage done	31 окт 2025 13:45	mariasmolina <mariaa.smolina@gmail.com>	2505e89
○	EditProfilePage done	30 окт 2025 21:34	mariasmolina <mariaa.smolina@gmail.com>	e97e1b8
○	ProfilePage and Database done	30 окт 2025 20:22	mariasmolina <mariaa.smolina@gmail.com>	83eb0f7
○	MainPage done	30 окт 2025 20:21	mariasmolina <mariaa.smolina@gmail.com>	bcd655c
○	Resources - Images and Fonts	30 окт 2025 16:47	mariasmolina <mariaa.smolina@gmail.com>	7c97994
○	AudioService and sounds	30 окт 2025 16:47	mariasmolina <mariaa.smolina@gmail.com>	67cde78
○	App logo and splash	30 окт 2025 16:46	mariasmolina <mariaa.smolina@gmail.com>	9fe366d
○	Language Localization	30 окт 2025 16:45	mariasmolina <mariaa.smolina@gmail.com>	6595722
○	Theme and Multi-language Localization	22 окт 2025 16:07	mariasmolina <mariaa.smolina@gmail.com>	ce61e16
○	Created repository	22 окт 2025 14:30	mariasmolina <mariaa.smolina@gmail.com>	7e0d182
○	Create README.md	22 окт 2025 14:24	Maria Smolina <mariaa.smolina@gmail.com>	b93561a
○	Initial commit	22 окт 2025 14:22	Maria Smolina <mariaa.smolina@gmail.com>	a88da05

LISA C. Figma Stiiliraamat



LISA D. Lokaliseerimisfailid AppResources

ResX Resource Manager

<